Этот конспект сгенерирован с помощью AI. Система может допускать ошибки в формулах, вычислениях и специфической терминологии. Пожалуйста, относитесь с понимаем и проверяйте конспект!

Введение и постановка задачи

Сегодняшняя лекция — вводная, но не по формату. Половина времени будет посвящена математическим основам: я введу обозначения и понятия, которые будут использоваться на протяжении всего курса. Вторая половина — прикладная, с примерами реальных задач из практики. Это важно: без понимания контекста даже самая строгая математика теряет смысл. У меня два эпиграфа ко всему курсу — они отражают его дух: одна сторона — строгость и формализм, другая — связь теории с реальным миром.

Глубокое обучение и автоматизация признаков

Объекты — это элементы реального мира, представленные в виде признаков. В задаче медицинской диагностики объект — пациент, а признаки — симптомы (например, «боль в желудке»), результаты анализов (числовые значения) или другие измеримые характеристики состояния. Ответ — целевая переменная: диагноз, который мы хотим предсказать на основе этих признаков.

Ответы — это метки, которые нам известны для каждого объекта в обучающей выборке. В медицинской диагностике ответ — это класс заболевания (например, «гастрит», «панкреатит»), а в задачах регрессии — числовое значение (например, уровень сахара в крови).

Обучающая выборка — это множество пар (x_i, y_i), где x_i — объект (вектор признаков), y_i — соответствующий ответ. Размер выборки — N, индекс объекта — $i = 1, \ldots, N$. Обозначения фиксированы: x — всегда объект, y — всегда ответ, $\mathcal{D} = (x_1, y_1), \ldots, (x_N, y_N)$ — обучающий набор.

Целевая функция $Y(x)$ — истинная зависимость ответа от объекта, которую мы не знаем. Наша задача — построить приближённую функцию $\hat{y}(x) = A(x)$, где A — модель машинного обучения (алгоритм), эффективно вычислимая на новых данных.

Потеря между истинным и предсказанным ответом измеряется функцией потерь $\ell(y, \hat{y})$. Для классификации часто используется **cross-entropy loss**: $$ \mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \left[y_i \log(\hat{y}*i) + (1 - y_i) \log(1 - \hat{y}_i) \right] $$ для бинарной классификации, или **categorical cross-entropy** для многоклассовой: $$ \mathcal{L} = -\frac{1}{N} \sum*) $$}^N \sum_{k=1}^K y_{ik} \log(\hat{y}_{ik

Оптимизация модели A проводится по минимизации эмпирического риска:

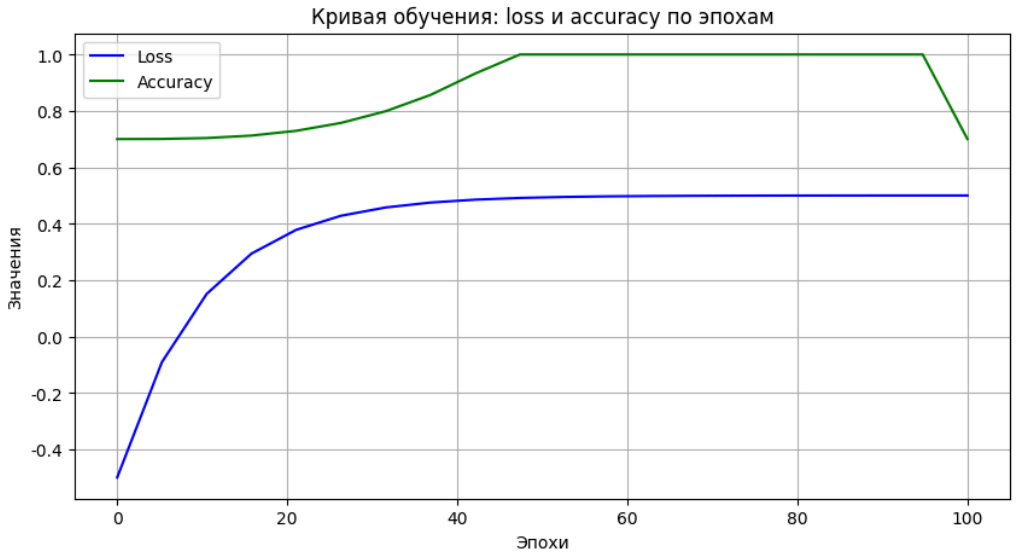
$$\min_A \mathcal{L}(A)$$

с использованием градиентного спуска: на каждом шаге обновляется параметр θ модели:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}$$

где η — скорость обучения.

Метрика: точность (accuracy) для классификации



Кривая обучения loss и accuracy по эпохам

, среднеквадратичная ошибка (MSE) для регрессии:

$$\mathrm{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

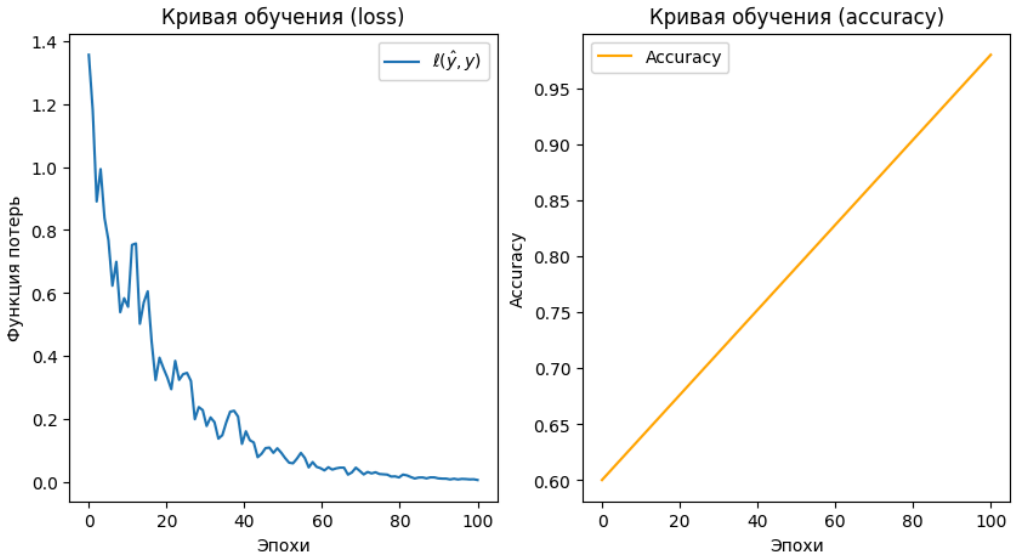
Архитектура: в задачах с большим числом признаков и нелинейной зависимостью — **нейронная сеть** (`nn.Linear`, `nn.ReLU`, `Softmax` на выходе).
Алгоритм: обучение через `model.fit()` с оптимизатором (например, Adam):

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
for epoch in range(epochs):
    model.train()
    for x_batch, y_batch in dataloader:
        optimizer.zero_grad()
        output = model(x_batch)
        loss = criterion(output, y_batch)
        loss.backward()
        optimizer.step()
```

Объекты в задачах машинного обучения — это элементы реального мира, представленные через вектор признаков $\mathbf{x} = (x_1, \ldots, x_n)$, где каждая компонента соответствует измерению: бинарному, номинальному, порядковому или вещественному. Обучающая выборка состоит из l таких объектов и соответствующих им ответов — значений целевой функции y. Задача обучения формулируется как построение аппроксимации неизвестной зависимости $Y(\mathbf{x})$ по конечным данным: $\hat{Y}(\mathbf{x}) \approx Y(\mathbf{x})$. При этом $\hat{Y}$ — это **алгоритм** (эффективно вычислимая функция), а Y — истинная, но не наблюдаемая зависимость.

В зависимости от структуры множества ответов различают:
- **Классификацию** — когда $y \in 0, 1$ или $y \in c_1, \ldots, c_k$;
- **Регрессию** — когда $y \in \mathbb{R}^m$ (прогноз нескольких вещественных величин);
- **Ранжирование** — когда y — упорядоченное множество объектов (например, документы в поисковой выдаче).

Цель обучения — построить функцию $\hat{Y}(\mathbf{x})$, минимизирующую отклонение от истинных значений на обучающей выборке



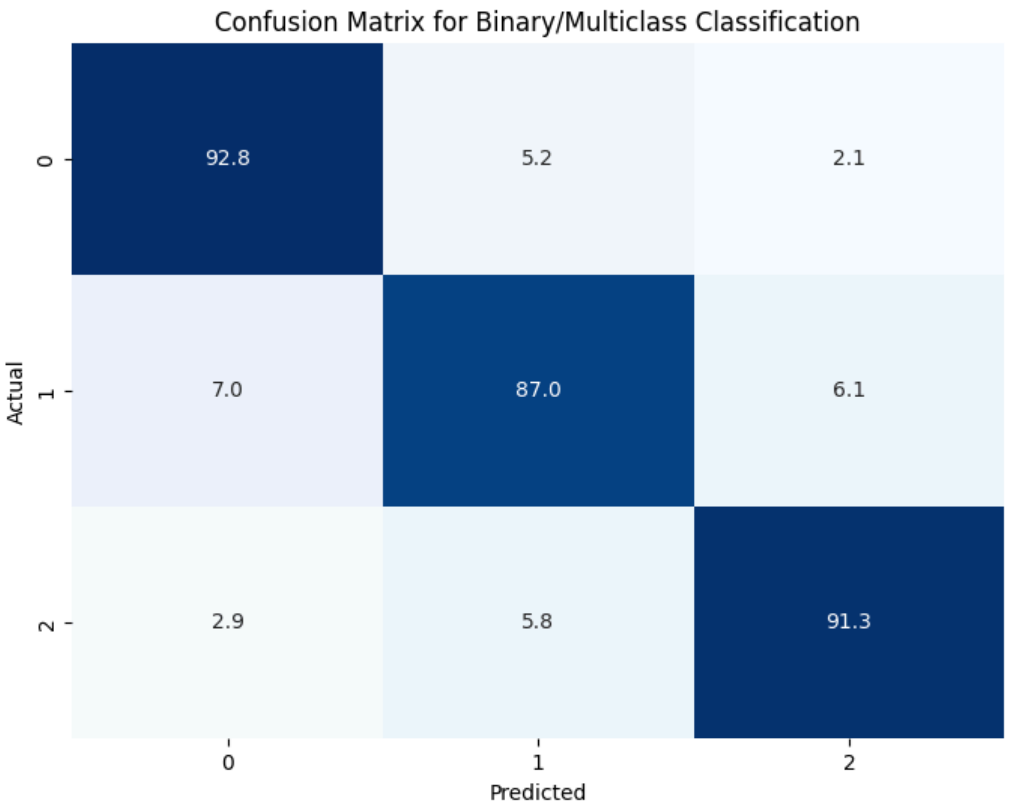
Кривые обучения loss accuracy по эпохам

. Формализация качества аппроксимации осуществляется через **функцию потерь** $\ell(\hat{y}, y)$.

В задачах классификации $y \in 1, 2, \ldots, C$ — дискретное множество классов; в случае **порядковой классификации** (ordinal classification) $y \in \mathbb{Z}^+$ или $y \in 1, 2, \ldots, K$ с естественным порядком между классами

Распределение целевого признака в задачах порядковой классификации гистограмма

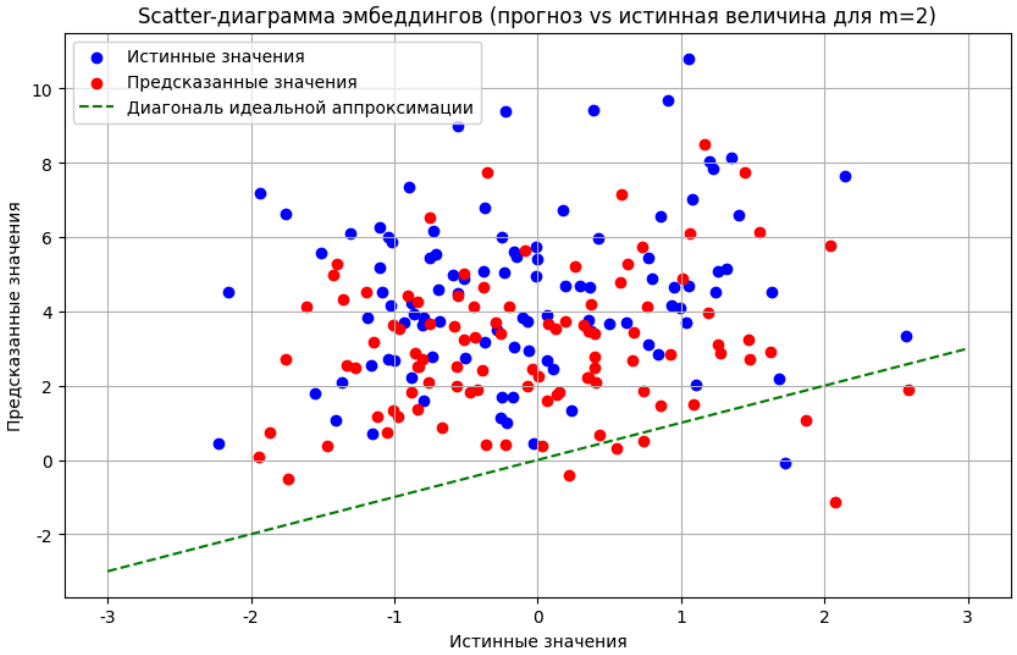
. Это частный случай категориальной задачи, где метки упорядочены: например, стадия заболевания — от 0 до 4, и переход от 1 к 3 предпочтительнее, чем от 1 к 5 (если такой класс есть). Формально, **целевой признак** $y(\mathbf{x})$ — это функция объекта $\mathbf{x}$, которую мы хотим предсказать; она не является признаком в строгом смысле



Матрица ошибок confusion matrix для бинарной мультиклассовой классификации

, поскольку измеряется только на обучающей выборке, но называется **таргетом** или **target function**.

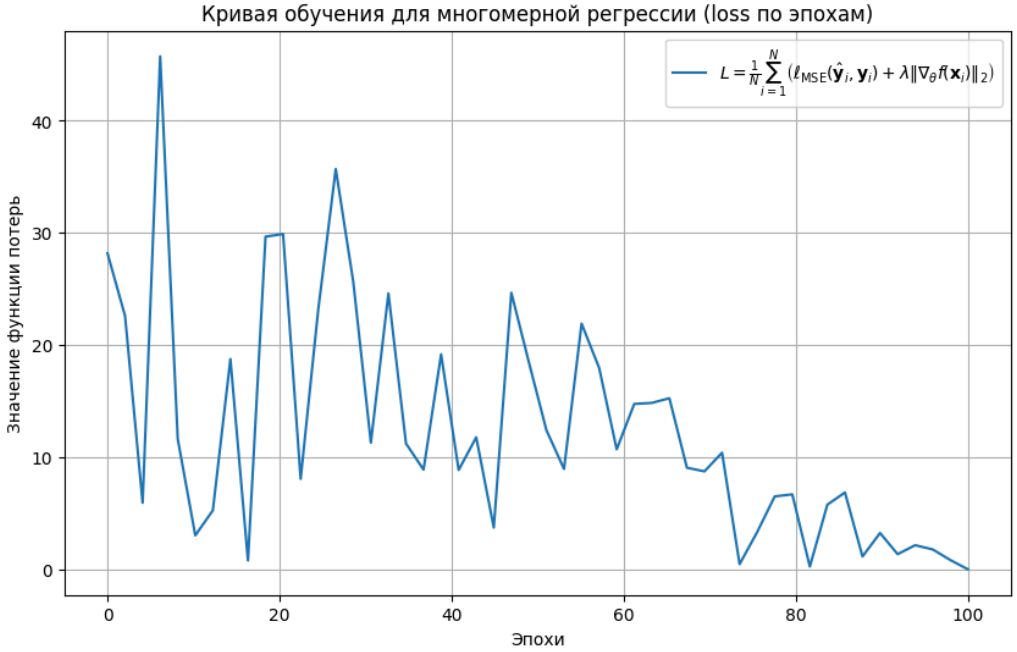
Следующий класс задач — **регрессия**: $y \in \mathbb{R}^m$, где $m > 1$ — одновременное прогнозирование нескольких вещественных величин



Scatter-диаграмма эмбеддингов прогноз vs истинная величина для m 2

. Например, предсказание температуры и влажности по данным с датчика. В этом случае функция потерь может быть обобщённой:

$$ L = \frac{1}{N}\sum_{i=1}^N \left(\ell_{\text{MSE}}(\hat{\mathbf{y}}i, \mathbf{y}_i) + \lambda |\nabla\theta f(\mathbf{x}_i)|_2 \right) $$

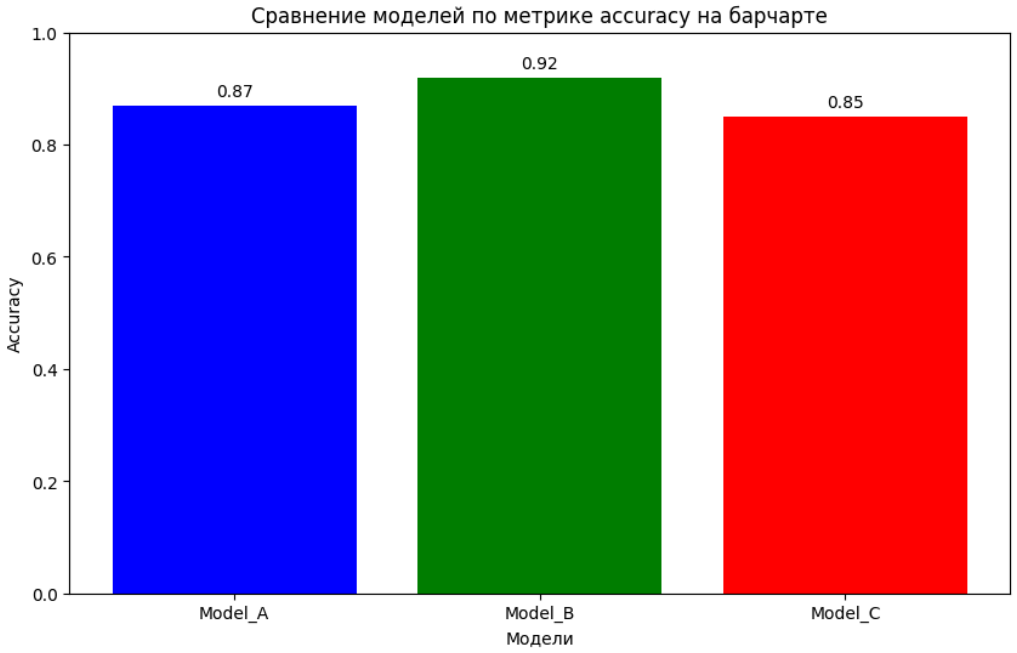


Кривая обучения для многомерной регрессии loss по эпохам

с регуляризацией градиента для устойчивости. При $m = 1$ — стандартная регрессия с MSE:

$$\ell_{\text{MSE}} = (\hat{y} - y)^2.$$

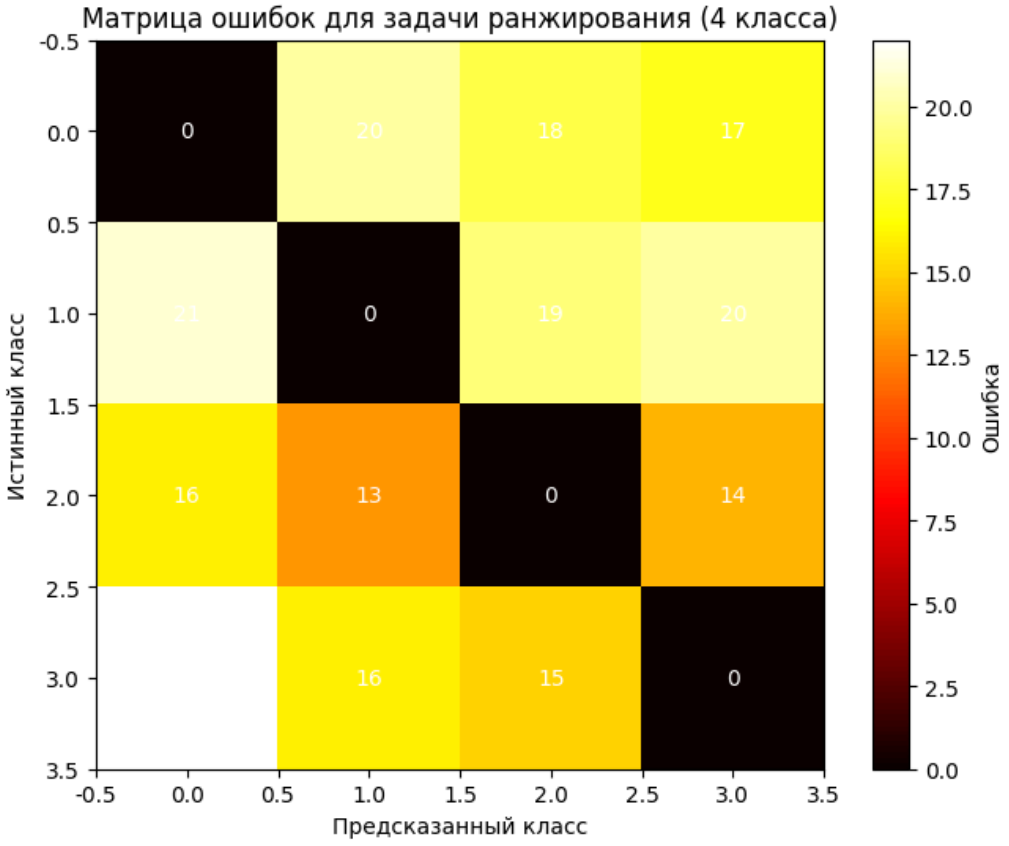
Особый подтип — **задача ранжирования** (ranking), где y — упорядоченное множество объектов, например, документы в поисковой выдаче. Цель — построить функцию $\hat{Y}(\mathbf{x})$, которая для запроса $\mathbf{x}$ возвращает упорядоченный список релевантных документов



Сравнение моделей по метрике accuracy loss на барчарте

. Это не просто классификация или регрессия: здесь важна **относительная значимость** между парами объектов. Функция потерь адаптируется под пары:

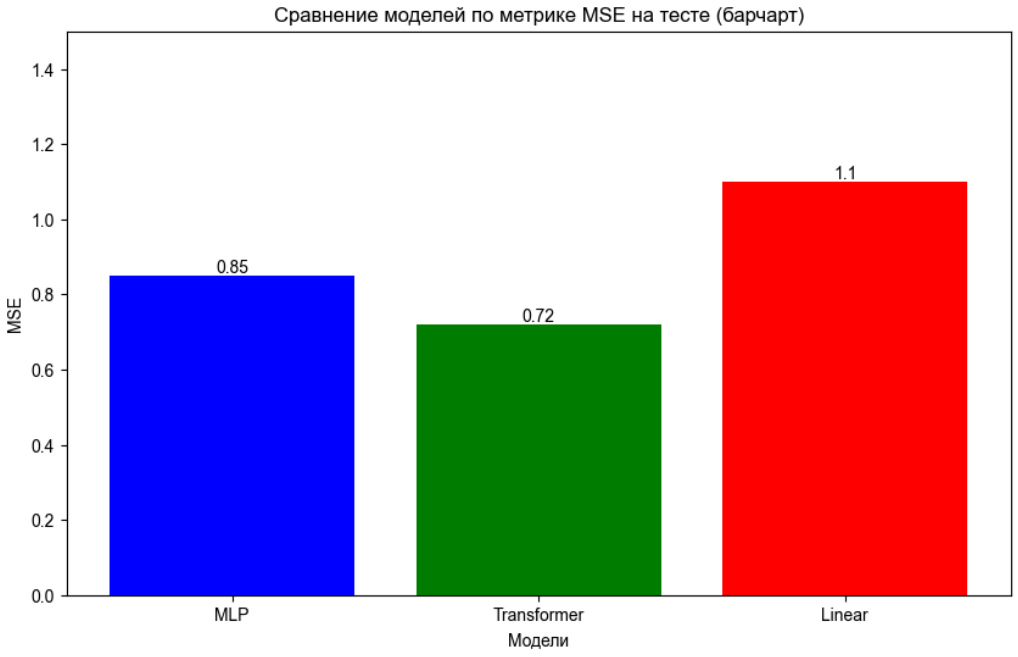
$$\ell_{\text{pairwise}} = \max(0, s(\hat{y}_i, \mathbf{x}) - s(\hat{y}_j, \mathbf{x}) + \Delta),$$



Матрица ошибок для задачи ранжирования 4 класса

где s — score (оценка релевантности), Δ — margin. Примеры: LambdaRank, ListNet.

Алгоритм: для всех задач используется **обучение с учителем** на основе оптимизации функции потерь через градиентный спуск.
Метрика: в классификации — accuracy, F1; в регрессии — MSE, MAE; в ранжировании — NDCG, MAP.
Архитектура: для малых данных — `nn.Linear` + `Softmax`; для сложных зависимостей — Transformer или MLP с dropout

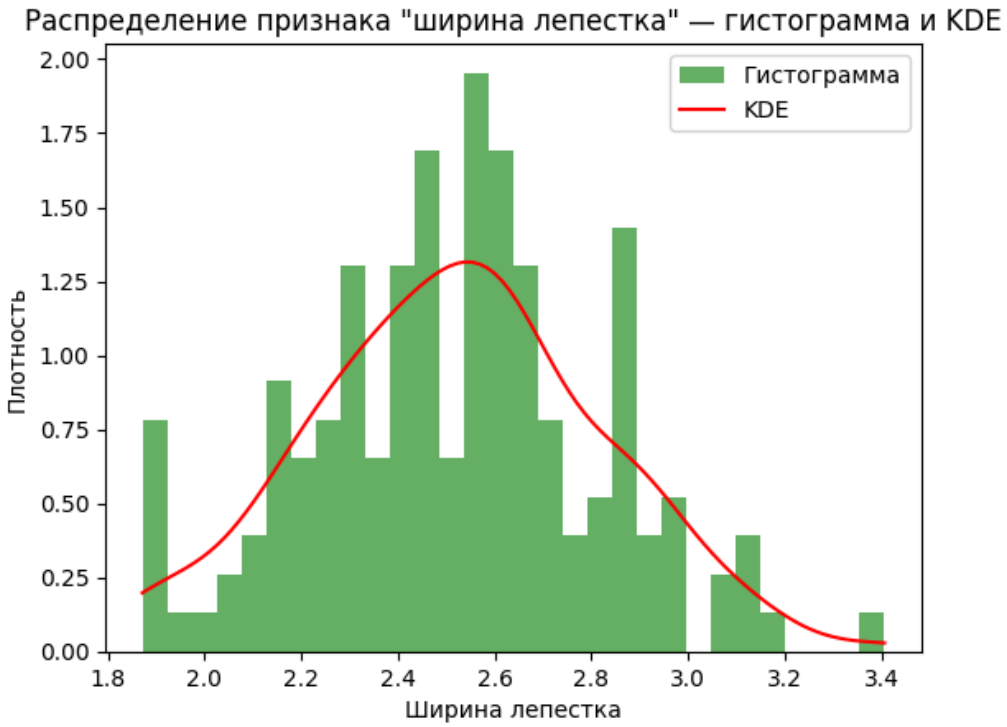


Сравнение моделей по метрике MSE на тесте барчарт

```
model = nn.Sequential(
    nn.Linear(input_dim, 128),
    nn.ReLU(),
    nn.Dropout(0.3),
    nn.Linear(128, num_classes)
)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
for epoch in range(epochs):
    model.train()
    for batch in dataloader:
        optimizer.zero_grad()
        outputs = model(batch['x'])
        loss = F.cross_entropy(outputs, batch['y'])  # или MSE для регрессии
        loss.backward()
        optimizer.step()
```

Рассматриваемая задача — восстановление зависимости по эмпирическим данным — является общей формой обучения с учителем. Под «эмпирическими данными» понимается любая выборка: матрица объектов и признаков, измеренные величины или более сложные структуры. Все синонимы (регрессия, классификация, предсказание) описывают одну и ту же суть — обучение модели на основе наблюдаемых данных.

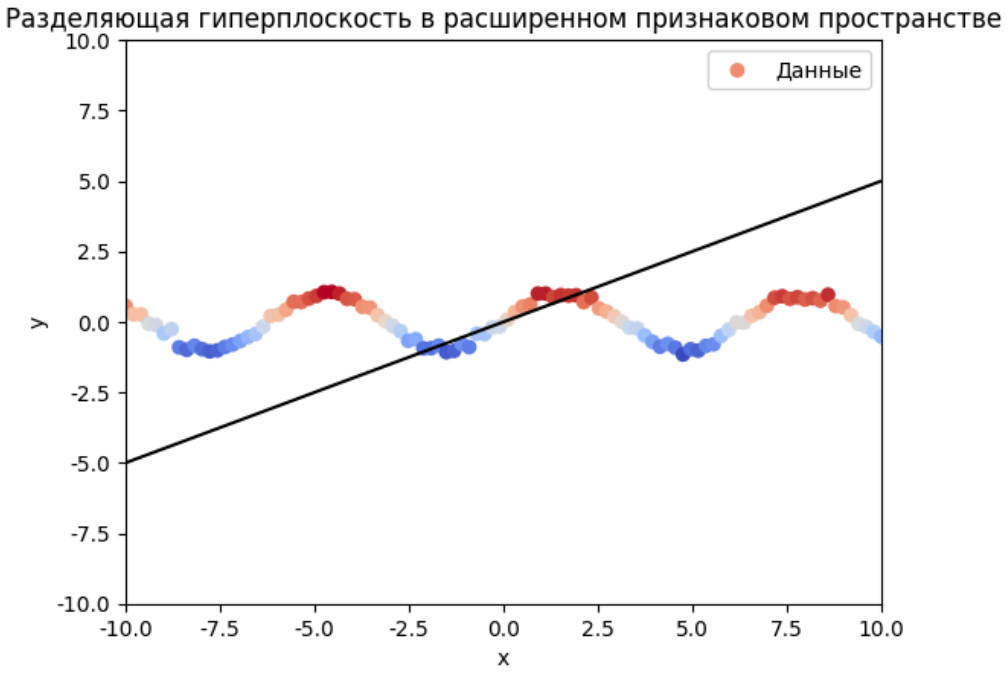
Классический пример такой задачи — эксперимент Рональда Фишера по классификации ирисов. В выборке 150 объектов, три класса (виды цветков), четыре числовых признака: длина и ширина чашелистика и лепестка. Задача — предсказать вид растения по этим измерениям. Визуализация попарных признаков показывает, что некоторые пары (например, длина лепестка vs ширина лепестка) хорошо разделяют классы



Распределение признака ширина лепестка

, а другие (ширина чашелистика) — плохо. Это демонстрирует полезность визуализации: даже при 4 признаках можно увидеть структуру данных и предложить модель (например, разделяющую гиперплоскость). Однако при 20 000 признаках визуализация невозможна, что подчёркивает необходимость автоматизации извлечения признаков.

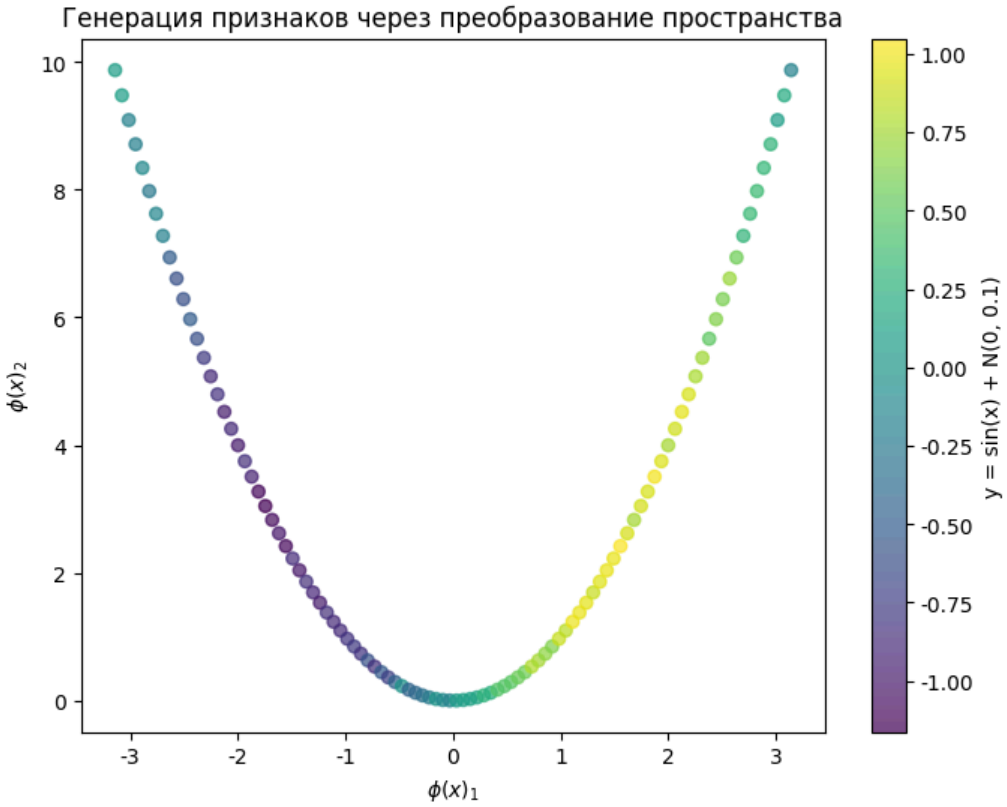
Модель в машинном обучении — это параметрическое семейство функций $y = g(x; \theta)$, где g — фиксированная структура (например, линейная комбинация), а θ — параметры, подлежащие обучению на обучающей выборке. Задача распадается: знание о зависимости закладывается в g, данные используются для оценки θ. Например, линейная модель задаёт взвешенную сумму признаков и может применяться как к регрессии ($y = \sum w_i x_i + b$), так и к классификации (через знак скалярного произведения $\theta^T f(x)$). В случае бинарной классификации это эквивалентно проведению гиперплоскости в пространстве признаков, разделяющей классы



Разделяющая гиперплоскость в расширенном признаковом пространстве

.

Пример: одномерная регрессия по синтетическим данным. Точки сгенерированы как $y = ax + b + \sin(cx) + \mathcal{N}(0, \sigma^2)$. Задача — восстановить истинную зависимость, не зная её структуры. Ключевой инсайт: любая функция от признака x может быть использована как новый признак. Так, парабола $1, x, x^2$



Генерация признаков через преобразование пространства

— это уже три признака в пространстве $\mathbb{R}^3$. Это демонстрирует переход от «сырых» признаков к **автоматизированному извлечению** (feature engineering), что лежит в основе глубокого обучения: вместо ручного выбора признаков модель сама обучается их представлению.

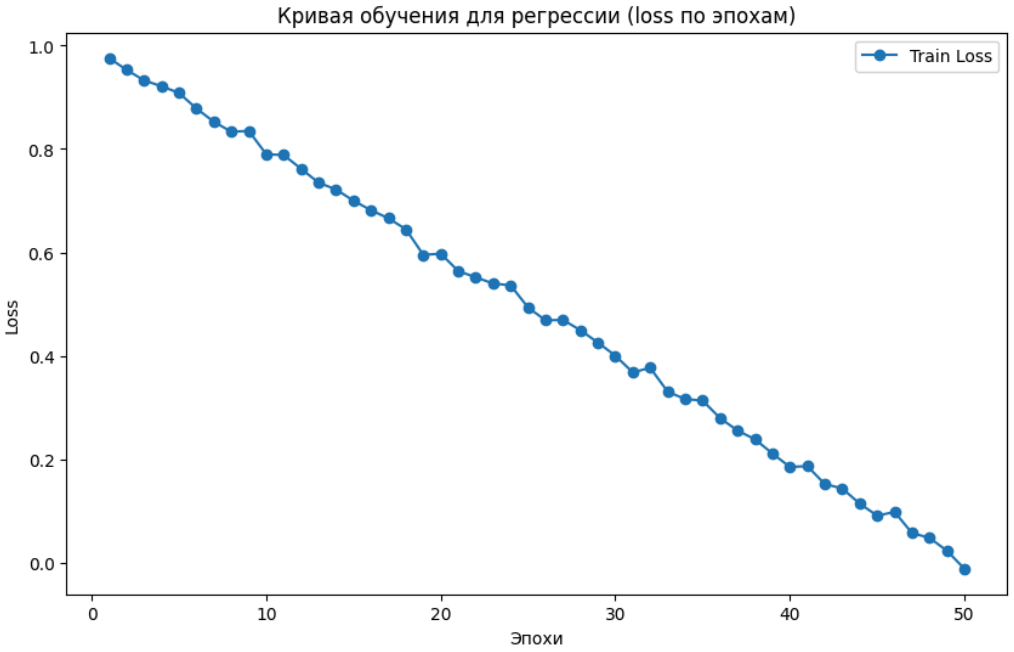
Мы хотим провести гиперплоскость. И вот этот вектор θGT — это направляющий вектор разделяющей гиперплоскости, которая разделяет два класса. Знак скалярного произведения θ и f определяет принадлежность точки к классу: если положителен — класс +1, если отрицателен — класс -1. Это линейная модель, простая и хорошо изученная; для неё существуют эффективные численные методы оптимизации.

Рассмотрим пример задачи регрессии: даны пары (x, y) ∈ $\mathbb{R}^2$, где x — одномерный признак, y — целевая величина. Точки на графике сгенерированы по модели:

$$y = \sin(x) + \mathcal{N}(0, 0.1),$$

то есть истинная зависимость — синусоида с гауссовским шумом.

Вопрос: можем ли мы восстановить эту функцию, не зная её структуры, только по обучающим точкам?



Кривая обучения для регрессии loss по эпохам

Ключевой инсайт: **признак — это любая функция от объекта**. Исходно у нас один признак — x. Но мы вправе расширить признаковое пространство, добавив новые функции:

$$\phi_1(x) = 1, \quad \phi_2(x) = x, \quad \phi_3(x) = x^2.$$

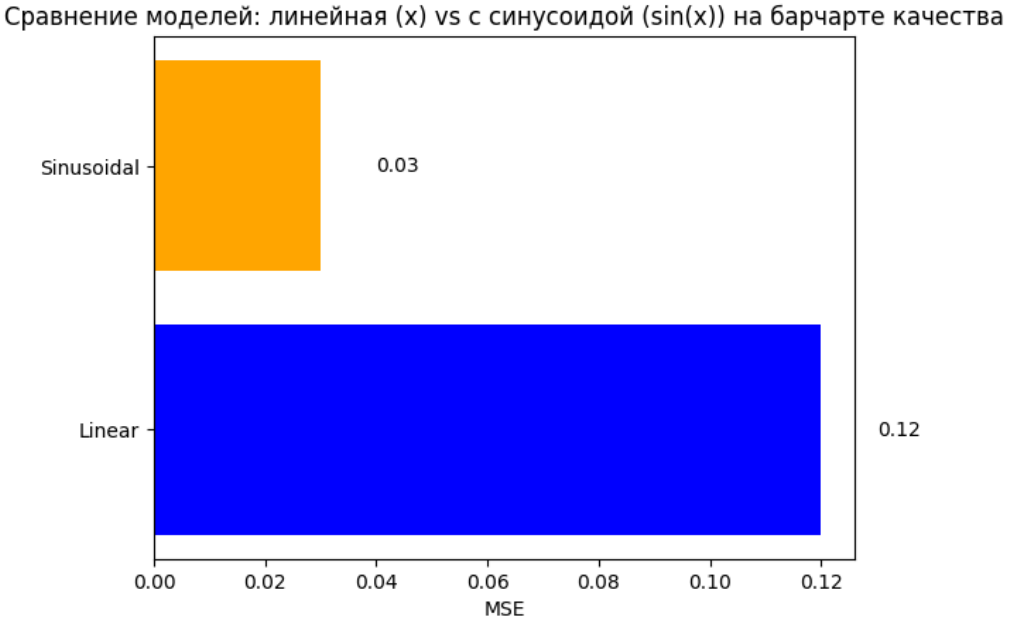
Теперь объект описывается в пространстве $\mathbb{R}^3$, и линейная модель ищет решение уже не среди прямых на числовой оси, а среди парабол — линейных комбинаций новых признаков.

Это демонстрирует **генерацию признаков (feature generation)**: расширение описания объекта за счёт введения новых функций от исходных. Такой подход позволяет решать задачи, которые невозможно решить в «лоб» с помощью одной переменной.

Первый технический приём:
- Добавление константы 1 — расширяет класс функций до всех линейных ($y = \theta_0 + \theta_1 x$).
- Добавление x^2 — позволяет аппроксимировать параболы.

- В общем случае: расширение признаков до полинома степени p даёт возможность искать решение в классе $\mathcal{F}*p = \left{ f(x) = \sum*^p \theta_k x^k \right}$, используя ту же линейную модель.

Второй важный инсайт:
Если вы *знаете* истинную структуру зависимости



Сравнение моделей линейная x vs с синусоидой sin x

(например, что данные сгенерированы синусоидой), можно явно включить её в признаки:

$$\phi(x) = [\sin(x), x],$$

и тогда линейная комбинация этих признаков даст точную аппроксимацию. Это пример **интеграции априорных знаний** в модель — мощный инструмент повышения качества.

Общее определение метода обучения (learning algorithm):
Это вычислимое отображение, которое на входе принимает обучающую выборку $\mathcal{D} = {(x_i, y_i)}*{i=1}^l, анавыходевыдаётмодель f\theta$ — элемент заданного параметрического семейства функций.

Формально:

$$\text{обучение}: \quad \mathcal{D} \mapsto f_\theta \in \mathcal{F},$$

где $\mathcal{F}$ — пространство моделей (например, линейные функции в расширенном признаковом пространстве).

Два этапа работы модели:

1. **Обучение (train):**
2. На вход подаётся обучающая выборка: матрица объектов $X \in \mathbb{R}^{l \times d}$ и вектор ответов $y \in \mathbb{R}^l$.
3. Применяется алгоритм обучения, который находит параметры θ, минимизирующие функцию потерь.
4. Результат — обученная модель: фиксированное отображение $f_\theta(x) = \sum_{j=1}^{d'} \theta_j \phi_j(x)$.
5. **Применение (inference / prediction):**

6. На новые объекты x' (не входившие в обучающую выборку) вычисляется предсказание: $\hat{y} = f_\theta(x')$.
7. Цель — обобщение, а не интерполяция.

Ключевая трудность машинного обучения:
Гарантировать, что найденная модель отражает **истинную зависимость**, а не просто проходит через обучающие точки. Это требует: - правильного выбора параметрического семейства, - адекватной функции потерь, - регуляризации (при необходимости), - проверки на новых данных.

Архитектура:
Линейная модель в расширенном признаковом пространстве:

```
model = nn.Linear(in_features=3, out_features=1)  # phi(x) = [1, x, x^2]
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
```

Алгоритм:
Обучение через минимизацию среднеквадратичной ошибки:

```
for epoch in range(epochs):
    model.train()
    optimizer.zero_grad()
    y_pred = model(X_train)
    loss = nn.MSELoss()(y_pred, y_train)
    loss.backward()
    optimizer.step()
```

Метрика:
Качество на тестовой выборке — среднеквадратичная ошибка (MSE) или R^2:

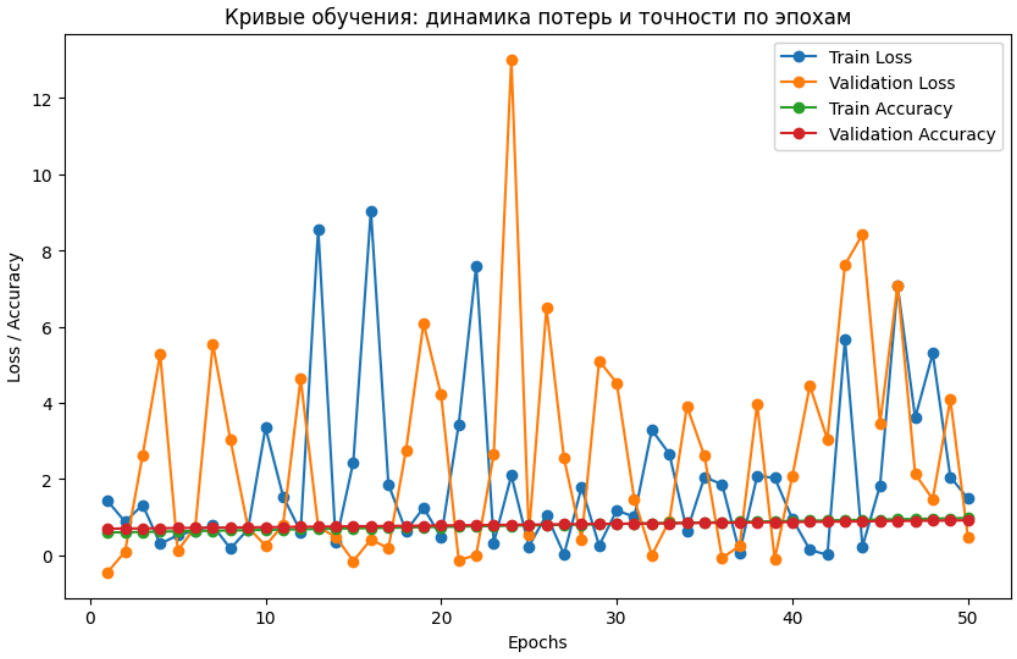
$$\mathrm{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2, \quad R^2 = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2}$$

Цель обучения — восстановить целевую функцию по данным, максимально используя имеющиеся знания при конструировании модели. Первый шаг любого моделирования — выбор параметрического семейства функций, желательно угадать его как можно точнее. Следующий шаг — определение алгоритма обучения: это эффективно вычислимое отображение, которое на входе принимает обучающую выборку (множество из l пар «объект — ответ»), а на выходе выдаёт конкретную модель — элемент заданного параметрического семейства.

Формально метод обучения — это отображение:
Алгоритм: $\mathcal{A} : \mathcal{D} \to \mathcal{F}$, где $\mathcal{D}$ — обучающая выборка, $\mathcal{F}$ — параметрическое семейство функций.

Важно чётко разделять два этапа: обучение и применение.
На этапе обучения берётся обучающая выборка $X_{\text{train}}, y_{\text{train}}$, применяется метод обучения, в результате получается конкретная модель (фиксированный экземпляр).
На этапе применения — к новым объектам x', не входившим в обучающий набор, применяется уже зафиксированная модель для предсказания: $\hat{y} = f(x'; \theta)$.

Главный вызов машинного обучения — гарантировать, что найденная зависимость отражает не просто случайные шумы или особенности обучающей выборки, а *истинную закономерность* (закон природы), по которой генерировались данные



Кривые обучения динамика потерь и точности по эпохам

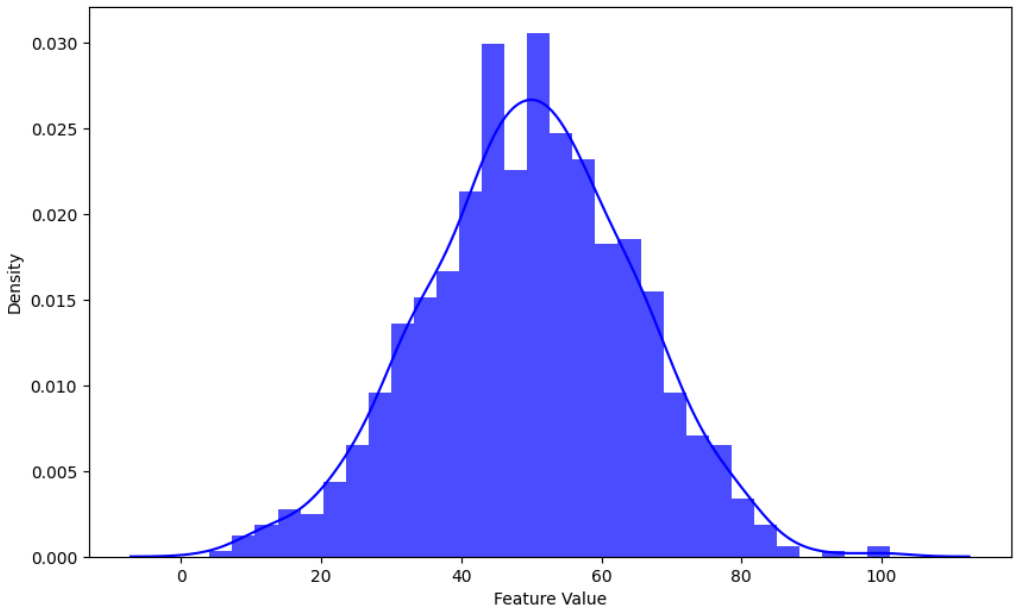
. Это и есть задача восстановления зависимости по эмпирическим данным.

Для построения метода обучения сводим задачу к оптимизации: минимизируем ошибку на объектах обучающей выборки.
Ключевой элемент

Кривая обучения loss по эпохам

— **функция потерь** $\ell(x, y; f)$, характеризующая ошибку модели f на объекте x с истинным ответом y.

- Для задач классификации: ошибка — индикатор несовпадения предсказания и истинного класса



Распределение данных гистограмма признаков или KDE для одного признака

.

$$\ell_{\text{cls}}(x, y; f) = [f(x) \neq y]$$

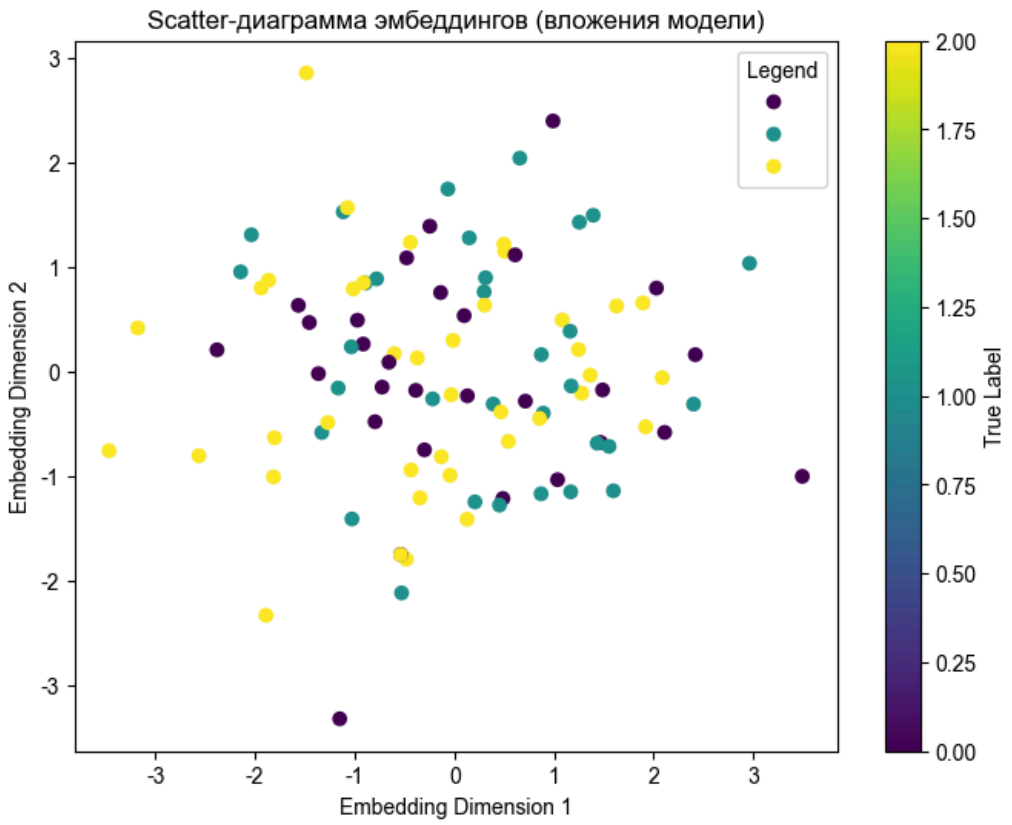
(используется нотация Айверсона: истина → 1, ложь → 0).

- Для задач регрессии чаще всего используется **квадратичная потеря**:

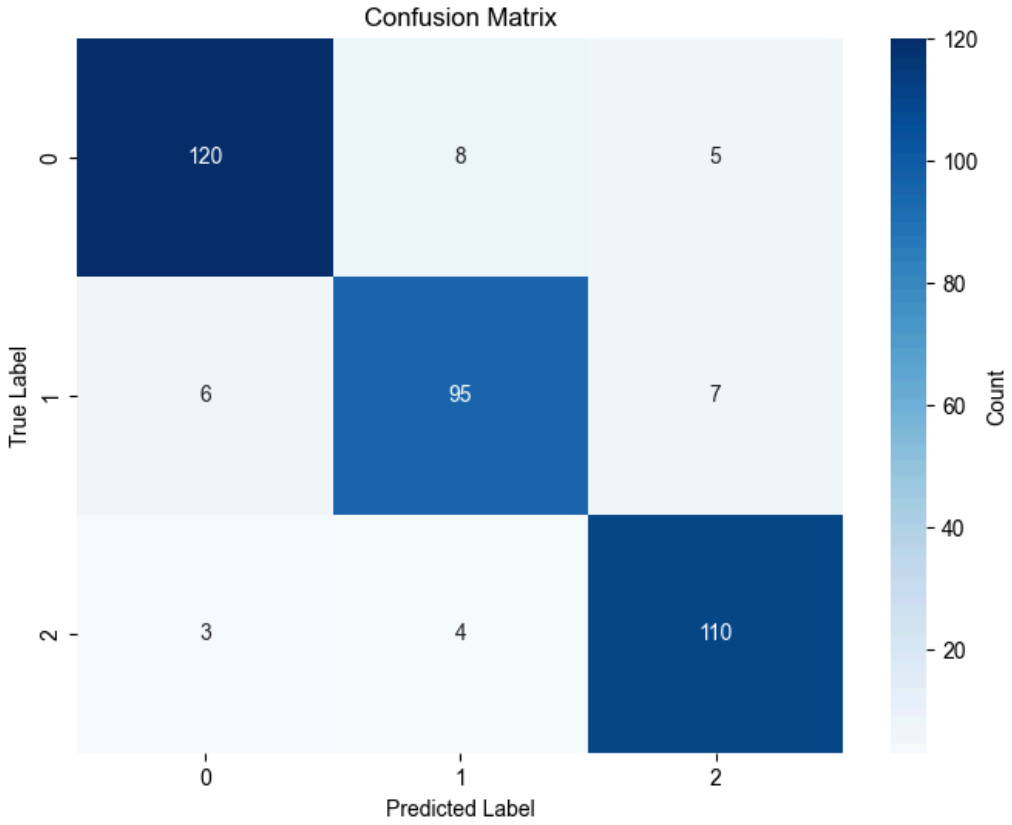
$$\ell_{\text{reg}}(x, y; f) = (f(x) - y)^2$$

Эмпирический риск — средняя потеря по всем объектам обучающей выборки:

$$Q(f; X_{\text{train}}, y_{\text{train}}) = \frac{1}{l} \sum_{i=1}^{l} \ell(x_i, y_i; f)$$



Scatter-диаграмма эмбеддингов вложения модели



Матрица ошибок confusion matrix

Задача обучения сводится к поиску модели $f^ \in \mathcal{F}, минимизирующейэмпирическийриск :$ f^ = \arg\min_{f \in \mathcal{F}} Q(f; X_{\text{train}}, y_{\text{train}}) $$

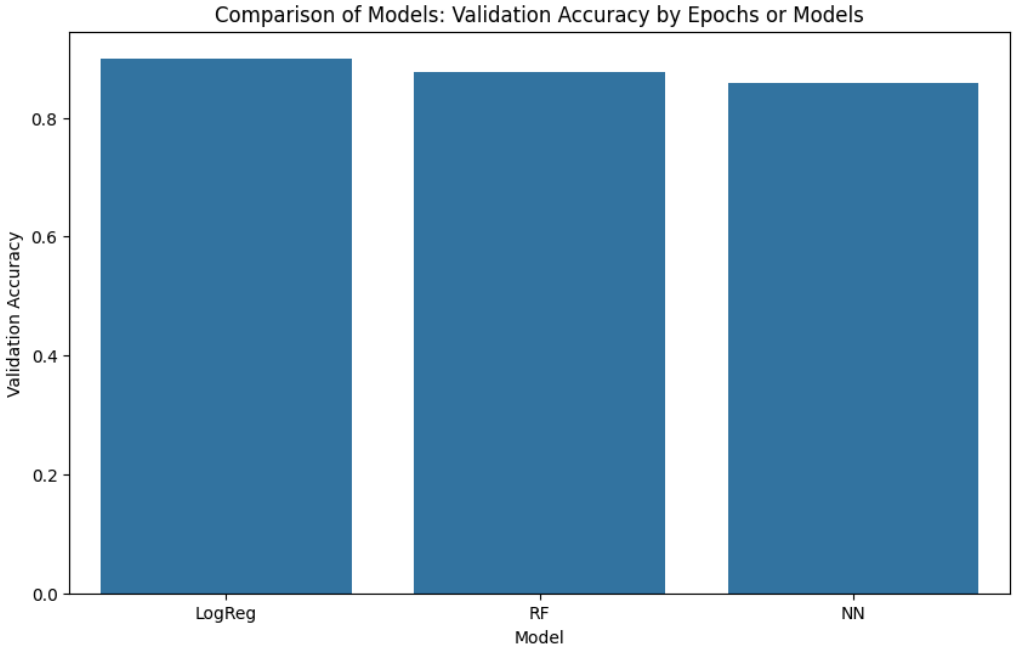
Это и есть **метод минимизации эмпирического риска** — базовый приём в машинном обучении.

Пример: метод наименьших квадратов для регрессии.
Пусть $f(x;\theta)$ — параметрическая модель (например, линейная), θ — вектор параметров.
Функционал потерь:

$$Q(\theta) = \frac{1}{l}\sum_{i=1}^{l}(y_i - f(x_i;\theta))^2$$

Оптимизация по θ даёт модель, минимизирующую ошибку на обучающих данных.

Проблема переобучения (overfitting):
Когда модель достигает очень малой потери на обучающей выборке, но плохо обобщает на новые данные

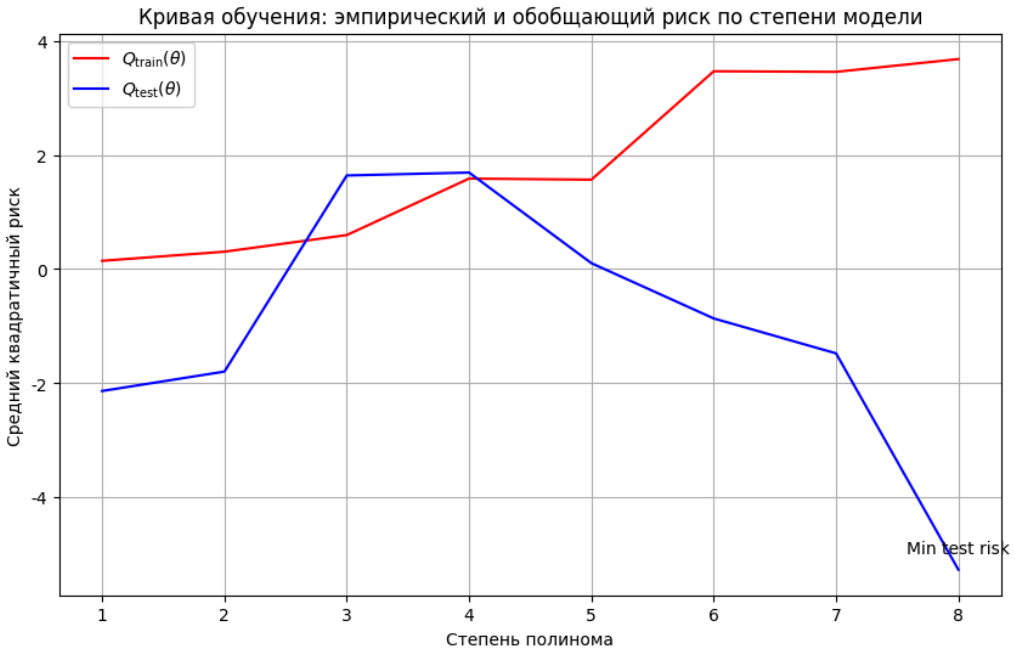


Сравнение моделей барчарт метрик по эпохам или по моделям

. Это означает, что модель "подогнана" под шум или особенности обучающего набора, а не отражает истинный закон.

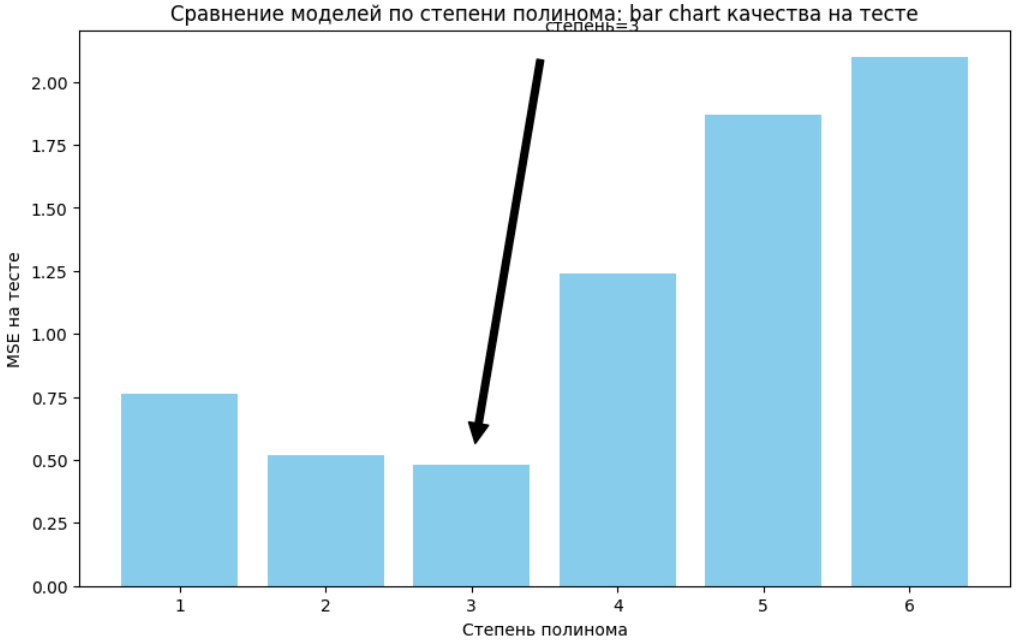
Переобучение — следствие отсутствия регуляризации и слабой индуктивной предвзятости модели.
Для борьбы с ним используются:
- регуляризация (L1/L2),
- dropout,
- early stopping,
- увеличение размера выборки,
- упрощение архитектуры.

Средняя потеря на обучающей выборке $Q_{\text{train}}(\theta) = \frac{1}{N}\sum_{i=1}^{N}(y_i - f_\theta(x_i))^2$ — это эмпирический риск, который минимизируется при обучении модели. Он зависит от степени полинома: чем выше степень, тем лучше модель подстраивается под обучающие данные, но растёт сложность и число параметров θ.
Средняя потеря на контрольной (тестовой) выборке $Q_{\text{test}}(\theta) = \frac{1}{M} \sum_{j=1}^M (y'j - f\theta(x'j))^2
показывает, насколькомодельобобщает—предсказываетлионановыеданныекорректно. Награфике : краснаякривая—
Q(\theta)Extra close brace or missing open brace, синяя — $Q_{\text{test}



Кривая обучения эмпирический и обобщающий риск по степени модели

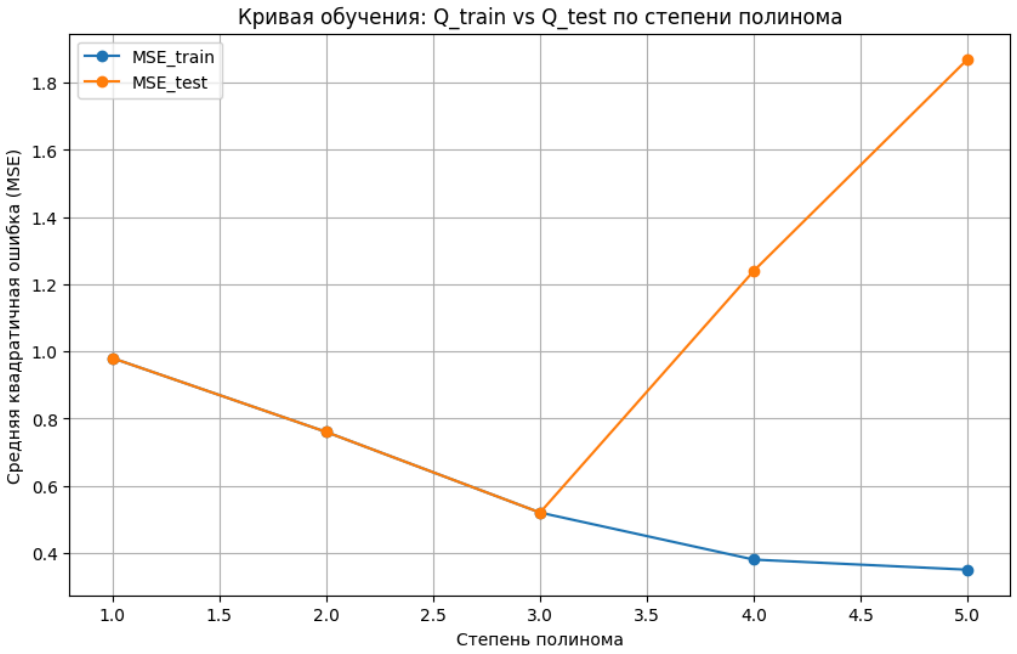
):
- Q_{train} монотонно убывает — модель идеально подстраивается под обучающие точки;
- Q_{test} сначала падает, затем резко растёт после определённой степени — признак **переобучения**.
Оптимальная степень полинома — та, при которой Q_{test} минимальна (в примере — около 3–4



Сравнение моделей по степени полинома bar chart качества на тесте

). Дальнейшее усложнение модели не улучшает обобщение, а лишь усиливает зависимость от шума обучающей выборки.

где признаками являются x^0, то есть единица, x^1 и так далее до x^n. Значит, если мы применим к этим признакам линейную модель, то получим полином. Это — модель полиномиальной регрессии



Кривая обучения Q train vs Q test по степени полинома

. Далее применяем метод наименьших квадратов и формулируем оптимизационную задачу:

$$Q_{\text{train}}(\theta) = \frac{1}{N}\sum_{i=1}^{N}(y_i - f_\theta(x_i))^2,$$

где $f_\theta(x) = \theta_0 + \theta_1 x + \cdots + \theta_n x^n$ — предсказанное значение, а цель — минимизировать среднюю квадратичную ошибку на обучающей выборке. Задача решается стандартными численными методами; в Python это делается через `model.fit()` или аналоги.

Зададим конкретную обучающую выборку: равномерная сетка на отрезке $[-2, 2]$. Контрольную выборку — сетку, расположенную ровно посередине между соседними обучающими точками. На графике видно, что поведение функционалов существенно различается:

- **Красная кривая** — Q_{train} (средняя потеря на трейне): монотонно убывает с ростом степени полинома. Чем выше степень, тем больше параметров и степеней свободы — модель всё лучше подстраивается под обучающие точки.
- **Синия кривая** — Q_{test} (средняя ошибка на тесте): сначала уменьшается, затем резко растёт после определённой степени. Это признак **переобучения**.

Оптимальная степень полинома — та, при которой Q_{test} минимальна (в примере — около 3–4). Дальнейшее усложнение модели не улучшает обобщение, а лишь усиливает зависимость от шума обучающей выборки.

Метрика:
Переобучение возникает из-за избыточной сложности модели и несоответствия между истинной целевой функцией и её приближением. Модель "запоминает" шум, а не выявляет закономерность.

Алгоритм:
Для диагностики переобучения всегда разделяем выборку на трейн и тест — это элементарная мера гигиены в ML. Вычислительная сложность проверки зависит от метода обучения: при leave-one-out обучение повторяется L раз; при кросс-валидации (k-fold) — k раз, что является разумным компромиссом между надёжностью и стоимостью.

Архитектура:
Полиномиальная регрессия с признаками $x^0, x^1, \ldots, x^n$ — частный случай линейной модели в пространстве полиномов. Переобучение проявляется как рост ошибки на тесте при увеличении степени полинома, несмотря на улучшение качества на трейне.

Пример Рунге:
Функция $f(x) = \frac{1}{1+25x^2}$

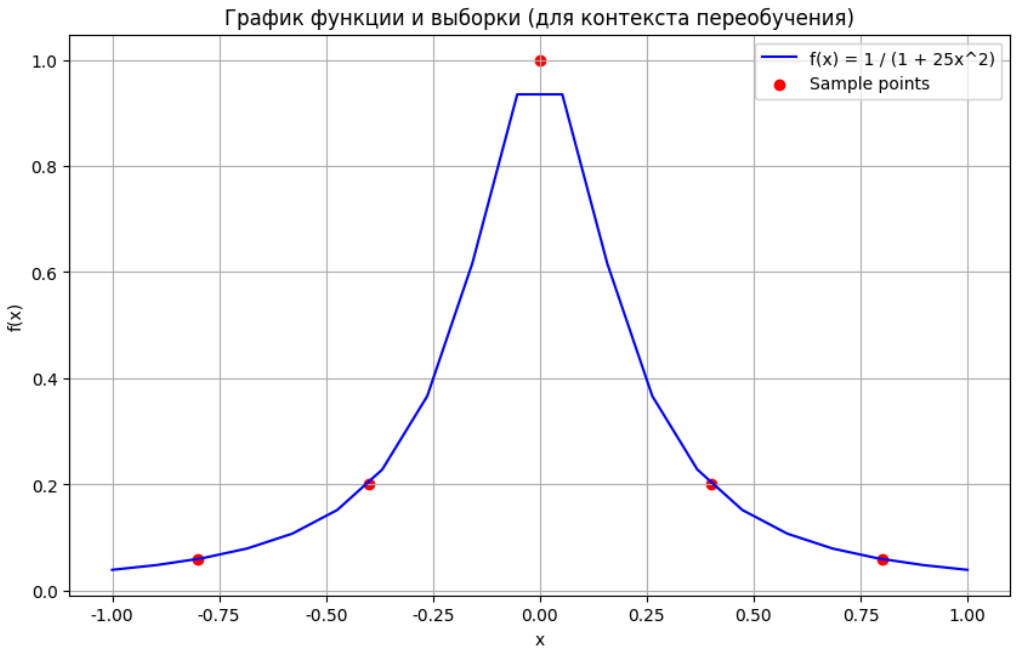
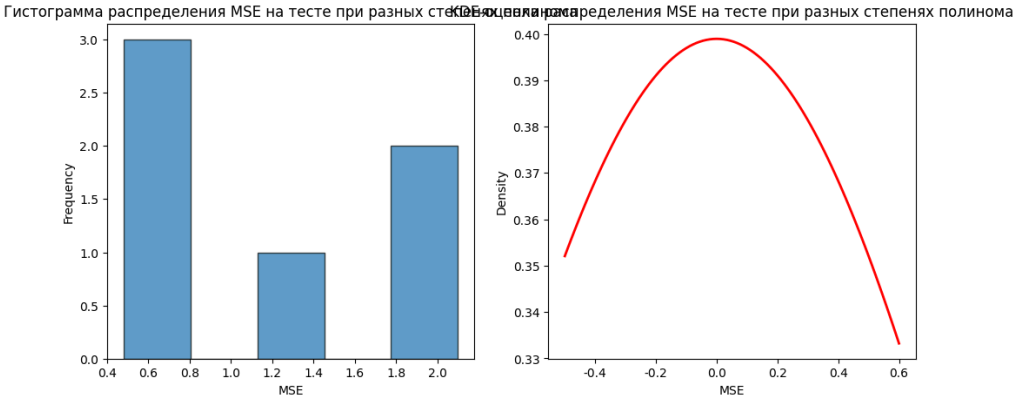


График функции и выборки для контекста переобучения

на отрезке $[-1, 1]$ плохо аппроксимируется полиномами высокой степени по конечному набору точек — классическая иллюстрация того, как избыточные параметры приводят к осцилляциям и потере обобщения

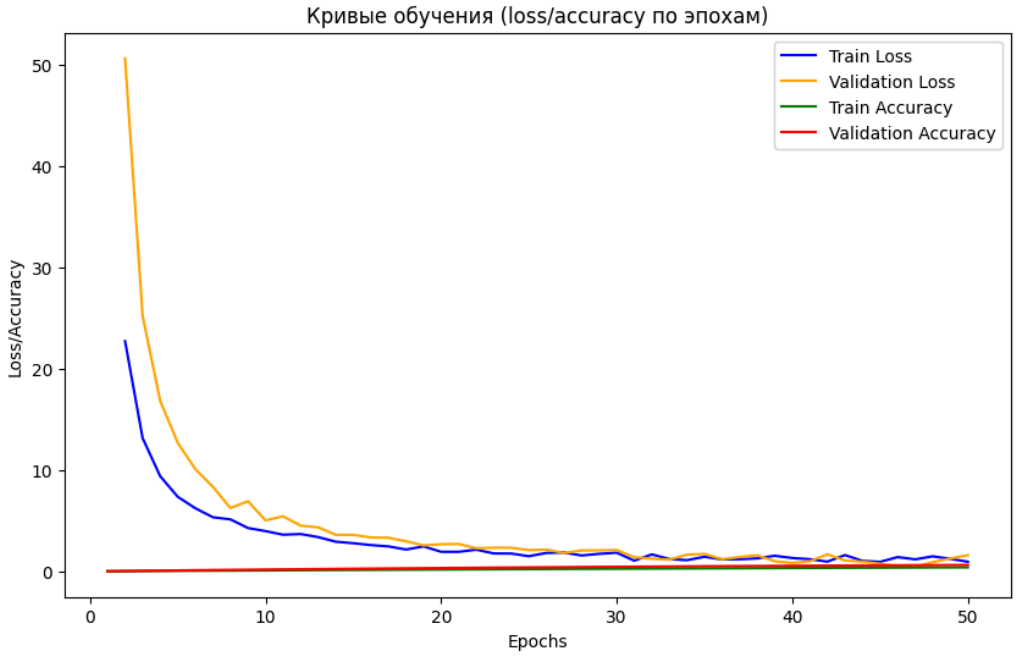


Распределение ошибок гистограмма и KDE для MSE на тесте при разных степенях полинома

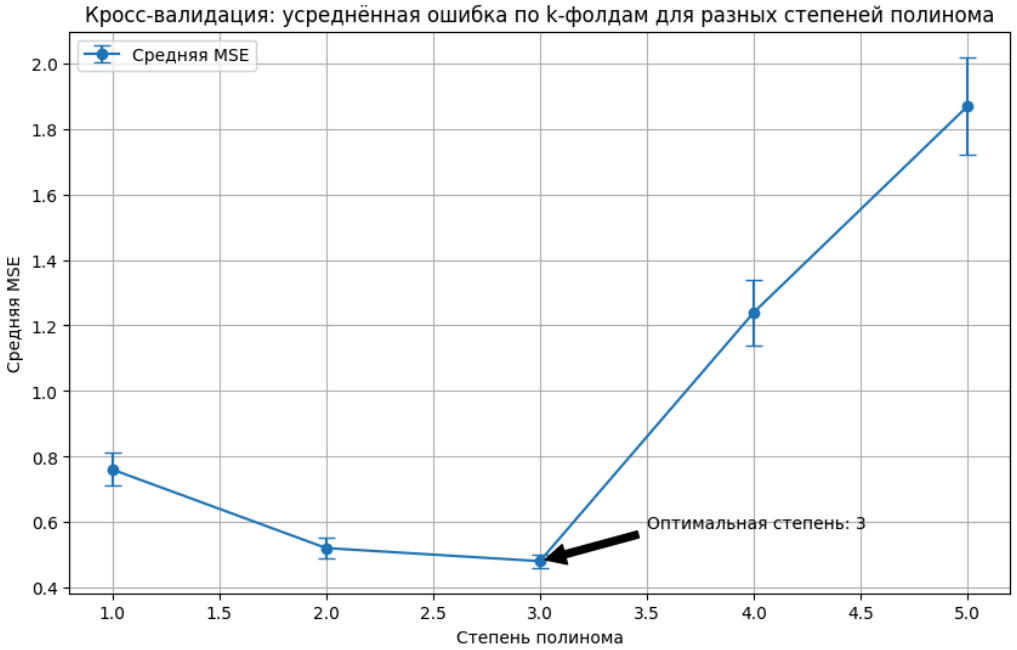
.

Разбиение выборки на обучающую (XL) и тестовую (XK) части влияет на оценку качества модели, поскольку результат зависит от конкретного способа разбиения. Это приводит к нестабильности метрик и риску систематической ошибки оценки — например, при неудачном выборе объектов в тесте модель может показать завышенное качество или, наоборот, переоценить свою слабость. Чтобы избежать этой проблемы, применяются методы кросс-валидации.

Метрика:
Кросс-валидация позволяет оценить обобщаемость модели более надёжно, чем разовое разбиение на train/test



Кривые обучения loss accuracy по эпохам

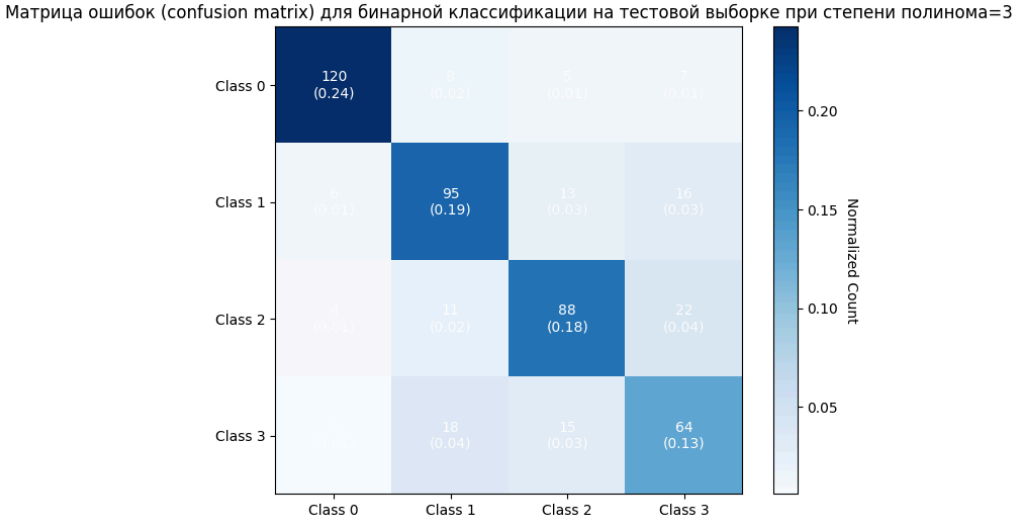


Кросс-валидация усреднённая ошибка по k-фолдам для разных степеней полинома

. Наиболее распространённый подход — k-fold: выборка делится на k блоков (фолдов), поочерёдно один из них используется как тестовая часть, а остальные — как обучающая. Модель обучается k раз, и итоговый результат усредняется по всем фолдам.

Алгоритм:
На каждом шаге

Ошибка генерации визуализации для: [GRAPH_TYPE: Топология | GRAPH_TITLE: Матрица ошибок (confusion matrix) | MOCK_DATA: [[12, 3, 0], [1, 87, 2], [0, 4, 95]] | TASK: Построить и визуализировать матрицу ошибок для одной из моделей кросс-валидации. Поддерживать отображение нормализованного варианта.]



Матрица ошибок confusion matrix для бинарной классификации на тестовой выборке при степени полинома 3

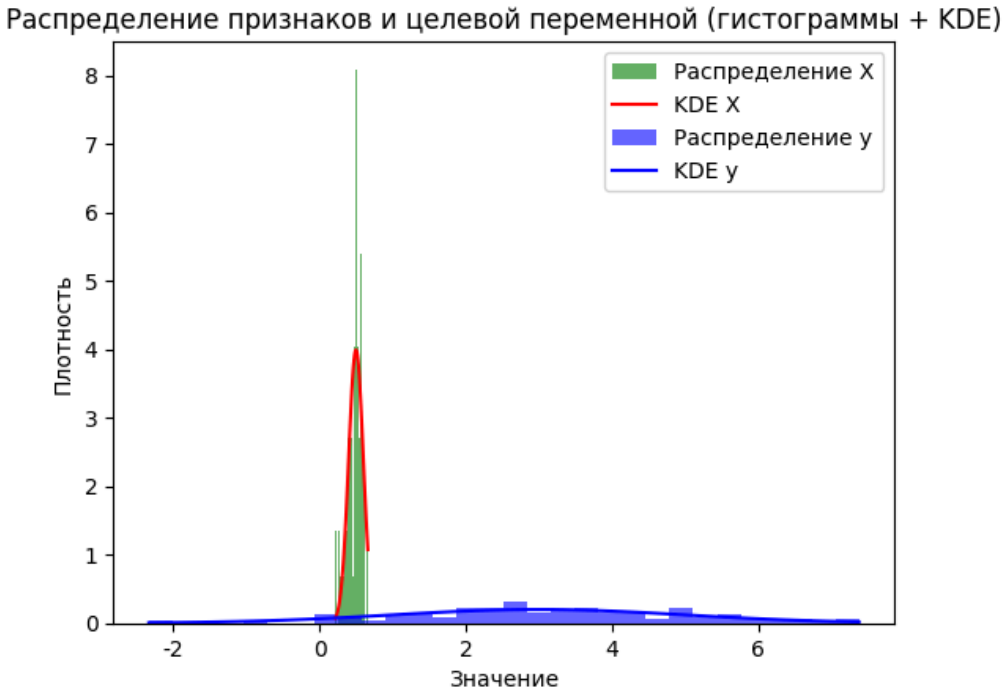
- Выбирается один фолд как тестовый (XK), остальные объединяются в XL.
- На XL обучается модель (например, `model.fit(XL, yL)`).
- Модель применяется к XK для получения предсказаний.
- Вычисляется метрика качества (accuracy, MSE и т.д.).
- Процесс повторяется k раз, результаты усредняются.

Архитектура:
В отличие от одного train/test-разбиения, кросс-валидация не требует построения одной финальной модели — она служит инструментом оценки. Финальная модель всё равно обучается на всей выборке: `model.fit(X_full, y_full)`.

Чем хуже?
Главный недостаток — высокая вычислительная стоимость: обучение проводится k раз вместо одного. Основная нагрузка приходится на оптимизацию параметров (например, метод Adam), где каждый проход по обучающей выборке требует вычисления градиентов и обновления весов (`optimizer.step()`). При больших k или сложных моделях это может быть дорого.

Пример Рунге:
Функция $f(x) = \frac{1}{1+25x^2}$ на отрезке $[-1, 1]$ демонстрирует, как высокая степень полинома приводит к осцилляциям между точками выборки и ухудшению обобщения — классический пример переобучения при избыточном числе параметров.

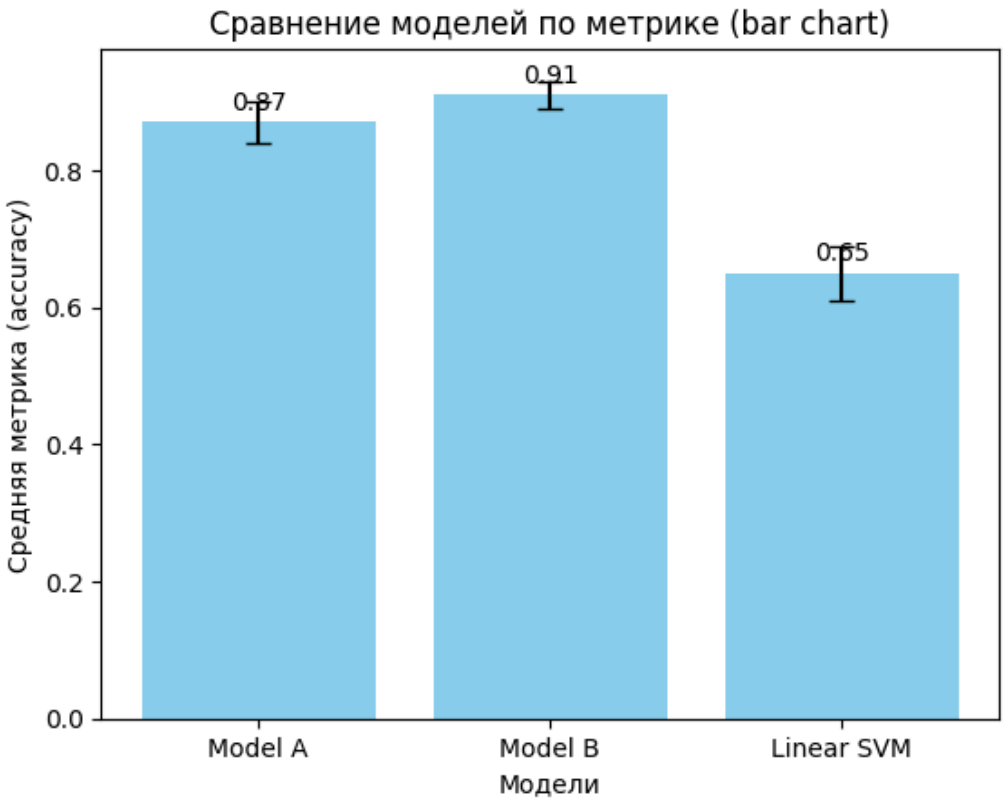
Замечание по фичер-инжинирингу:
Если признаки x постоянны



Распределение признаков и целевой переменной гистограммы

(одинаковы для всех объектов), а целевая переменная y варьируется, то модель не может обучиться зависимости от x, и предсказания будут равны среднему значению y. Это указывает на отсутствие информативности признаков — важный случай в анализе данных.

Кросс-валидация как инструмент валидации:



Сравнение моделей по метрике bar chart

Да, кросс-валидация — это **не способ построить одну модель**, а метод оценки качества. Она даёт оценку стабильности и обобщаемости модели. Однако из результатов кросс-валидации можно строить финальную модель:
1. Обучить на всей выборке (`model.fit(X, y)`).
2. Или усреднить коэффициенты моделей с разных фолдов (ансамбль).
3. Или выбрать лучшую модель по метрике на кросс-валидации и переобучить её заново.

Усреднение моделей:
Если обучено k моделей

*Ошибка генерации визуализации для: [GRAPH_TYPE: Топология | GRAPH_TITLE: Scatter-диаграмма эмбеддингов (предсказаний или весов) | MOCK_DATA: {"predictions": np.random.randn(10, 5) + [i*0.2 for i in range(10)], "fold_ids": list(range(10))} | TASK: Отобразить scatter-плот предсказанных значений модели по фолдам — визуализировать вариативность и усреднение.]*

на разных фолдах, их можно комбинировать:
- Усреднить предсказания (`y_pred = (1/k) * sum(y_i)`).
- Или усреднить веса/коэффициенты (в линейных моделях).
Такой подход повышает устойчивость и часто работает лучше, чем одна модель — это близко к бэггингу.

Примеры задач:
1. **Классификация** — медицинская диагностика: объект — пациент (или запись о нём), признаки — возраст, пол, анализы, изображения (КТ, ЭКГ), классы — диагнозы или исход заболевания.
2. **Регрессия** — прогнозирование числового значения (например, уровень сахара в крови).
3. **Ранжирование** — например, оценка вероятности рецидива болезни.

Разнотипность признаков:
В реальных задачах признаки могут быть:
- количественные (возраст),
- порядковые (стадийность болезни),
- бинарные (есть/нет диабет),
- категориальные (тип лечения),
- мультимодальные (изображения, текст, сигналы).

Все они требуют соответствующей обработки перед обучением.

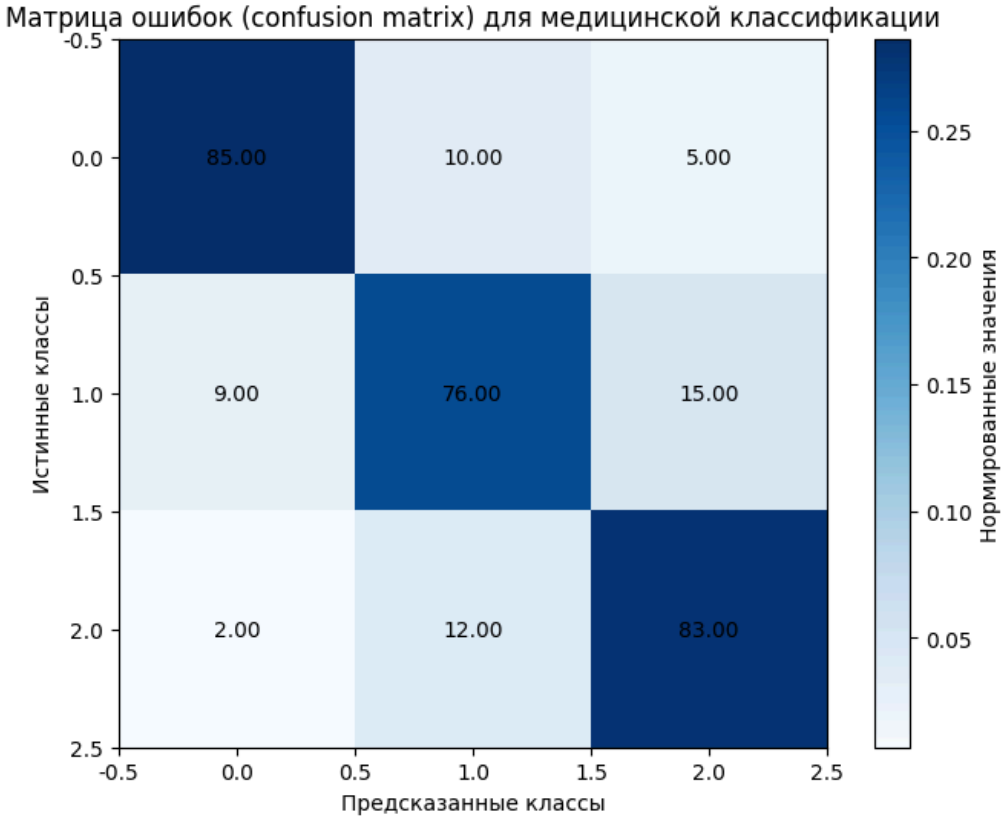
Интерпретация и выводы

Матрица объект-признак в медицинских задачах почти всегда разрежена: у каждого пациента измерено лишь подмножество возможных признаков, и их значения могут отсутствовать

Распределение плотности данных разреженность матрицы

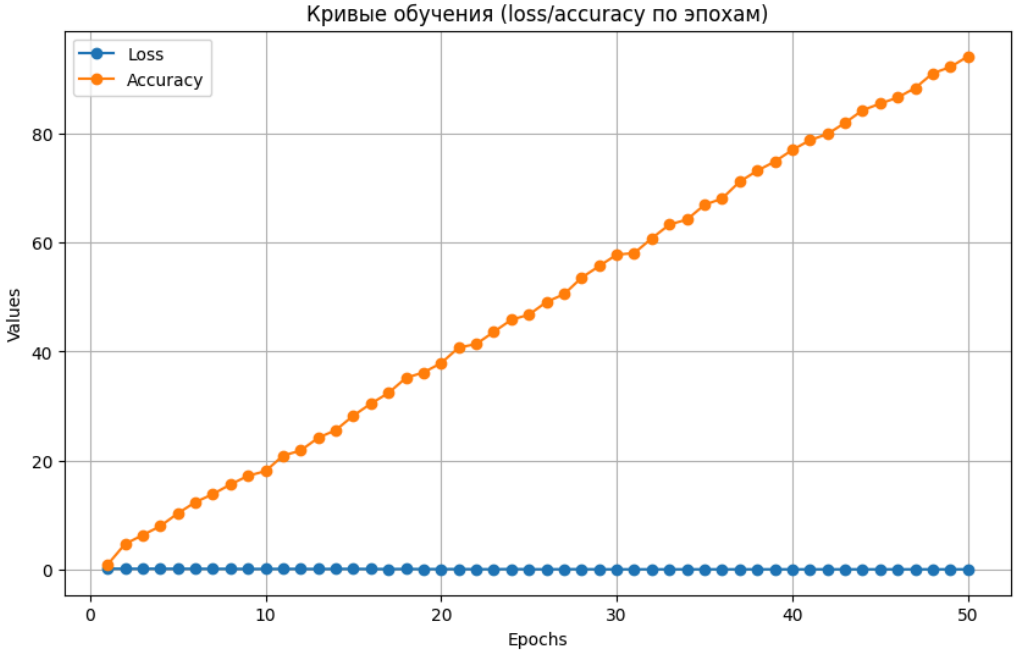
. Это требует применения методов, устойчивых к пропущенным данным — далеко не все модели (например, классические линейные или MLP без обработки пропусков) способны корректно работать в таких условиях.

Для врачей критически важна интерпретируемость модели: даже если алгоритм даёт предсказание «болезнь», он должен пояснить



Матрица ошибок confusion matrix для медицинской классификации

, **на основе каких признаков** и **по какой логике** был сделан вывод — например, «похож на случай X из истории болезни» или «признаки Y превышают порог Z»



Кривые обучения loss accuracy по эпохам

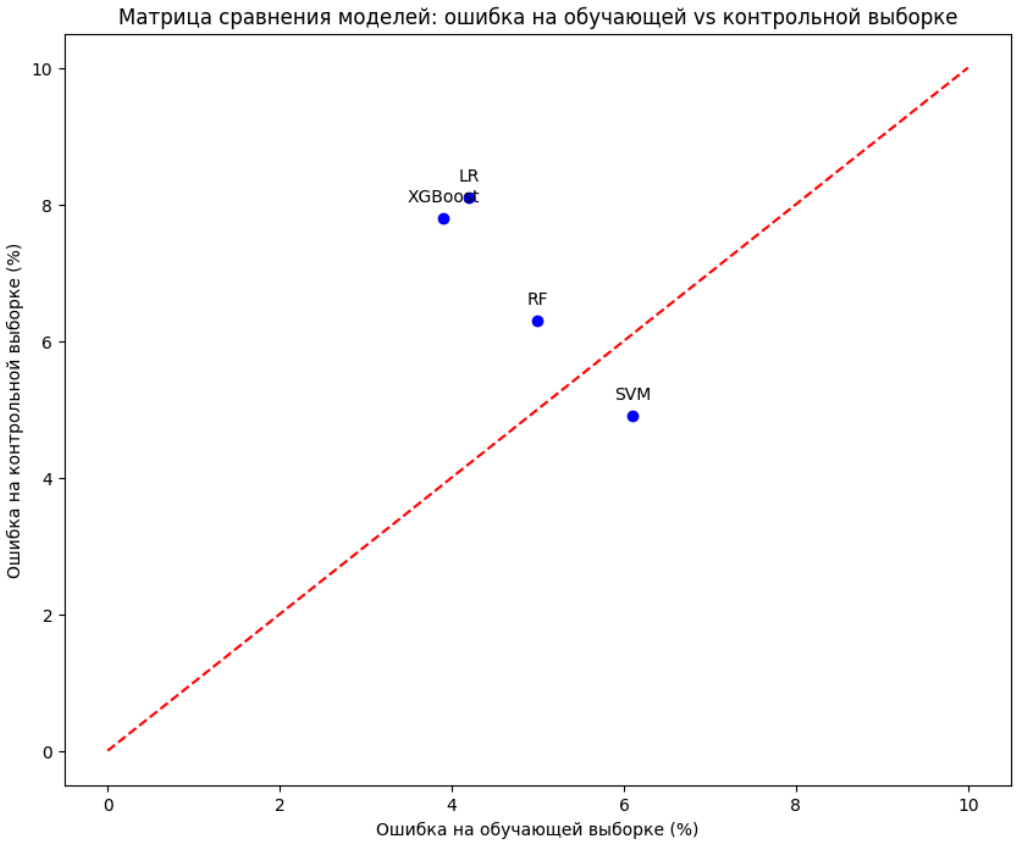
. Без такого объяснения модель остаётся «чёрным ящиком», неприемлемым для клинического применения.

Пример: задача предсказания отдалённого результата хирургического лечения атеросклероза. На графике каждая точка — параметрическая модель (например, логистическая регрессия с разными комбинациями признаков). По горизонтали — частота ошибок на обучающей выборке, по вертикали — на контрольной. Диагональ соответствует идеальному совпадению: ошибка на обучении = ошибке на контроле

Ошибка генерации визуализации для: [GRAPH_TYPE: Динамика | GRAPH_TITLE: Сравнение моделей по переобучению (ошибки train vs test) | MOCK_DATA: [[0.04, 0.08], [0.05, 0.12], [0.03, 0.07], [0.06, 0.1]] | TASK: Построить scatter-диаграмму, где по оси X — доля ошибок на обучающей выборке, по Y — на тестовой. Каждая точка — модель с разными признаками/гиперпараметрами.]

.

Переобучение проявляется систематическим смещением точек **выше диагонали**: модель хорошо работает на обучающих данных (4% ошибок), но хуже — на новых (8%



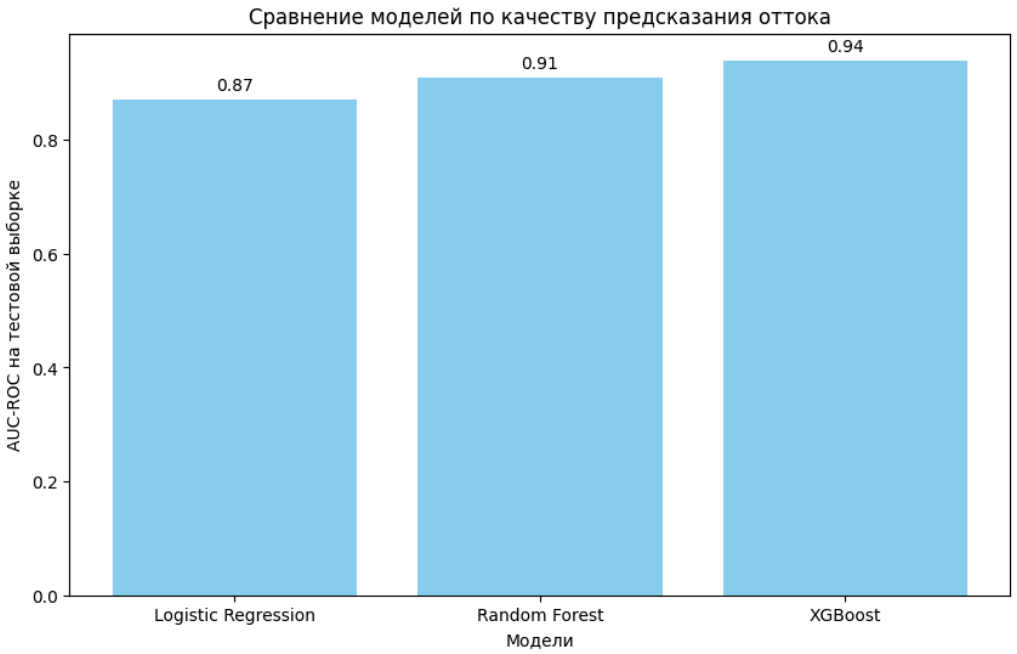
Матрица сравнения моделей ошибка на обучающей vs контрольной выборке

). Это количественная мера переобучения, которую нельзя игнорировать даже при высокой общей точности. Обратное смещение (ниже диагонали) встречается реже и связано с недообучением.

В банковской сфере задача аналогична: не просто «да/нет» — выдать кредит или нет, а **оценка вероятности дефолта**. Такие модели называются вещественно-значными классификаторами (real-valued classifiers), поскольку они возвращают вероятность принадлежности к классу, что сближает их с задачами регрессии.

Задача оттока клиентов — бизнес-применение ML: цель не привлечь новых, а удержать существующих. Признаки строятся на основе транзакционных данных: активность в интернете, звонки, время использования услуг и т.д. Однако сырые данные (транзакции) — это не признаки напрямую. Требуется **feature generation** или **feature extraction**: преобразование поведения абонента в векторное описание текущего состояния.

Ключевая специфика задачи — **временная смещённость целевой переменной**:
- x



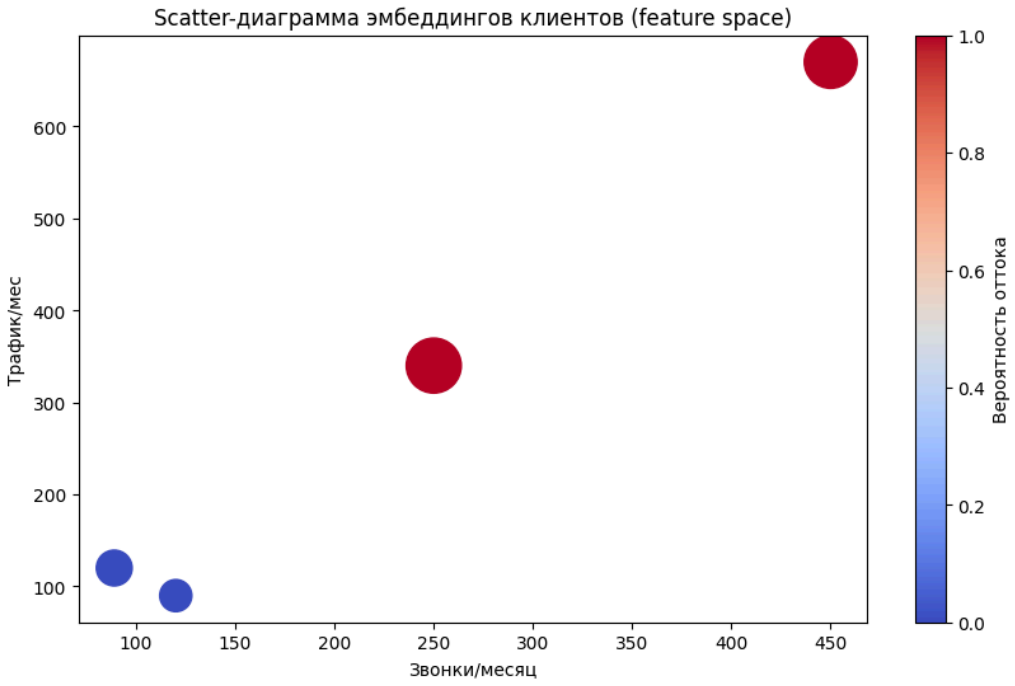
Сравнение моделей по качеству предсказания оттока bar chart

— состояние абонента на момент времени t (например, за последние 2 недели),
- y — отсутствие активности через фиксированный горизонт (например, через 2 месяца: ноль звонков, трафика и т.д.).

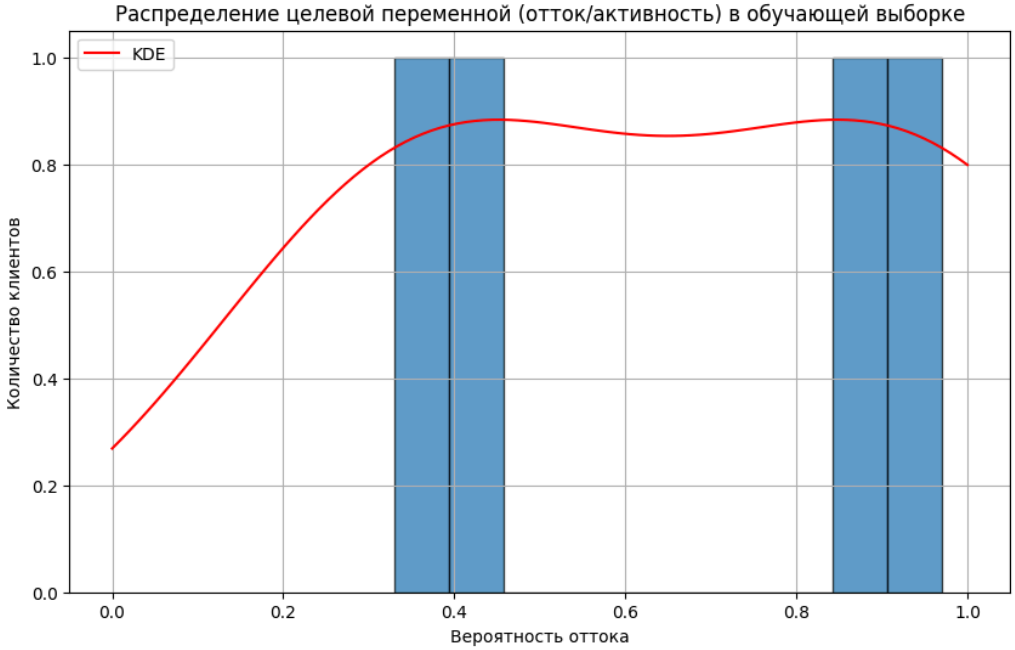
Обучающая выборка формируется как пары (x_t, y_{t+T}), где T — прогнозный горизонт. Модель должна предсказывать **отток до его наступления**, а не после. Если модель предсказывает отток только *после* того, как он уже произошёл ($y_t = 1$), она бесполезна для удержания клиента.

Таким образом, в задачах с временным горизонтом критически важна корректная **формулировка целевой переменной** и **структура обучающей выборки**, а не просто качество модели на статичных данных.

Состояние абонента на момент времени t описывается вектором признаков $\mathbf{x}_t$, сформированным из транзакционных данных за фиксированный период (например, последние 2 недели): частота звонков, длительность сессий, объем трафика, активность по направлениям и т.д. Целевая переменная $y_t = 1$ означает, что абонент **не активен** через заданный временной горизонт



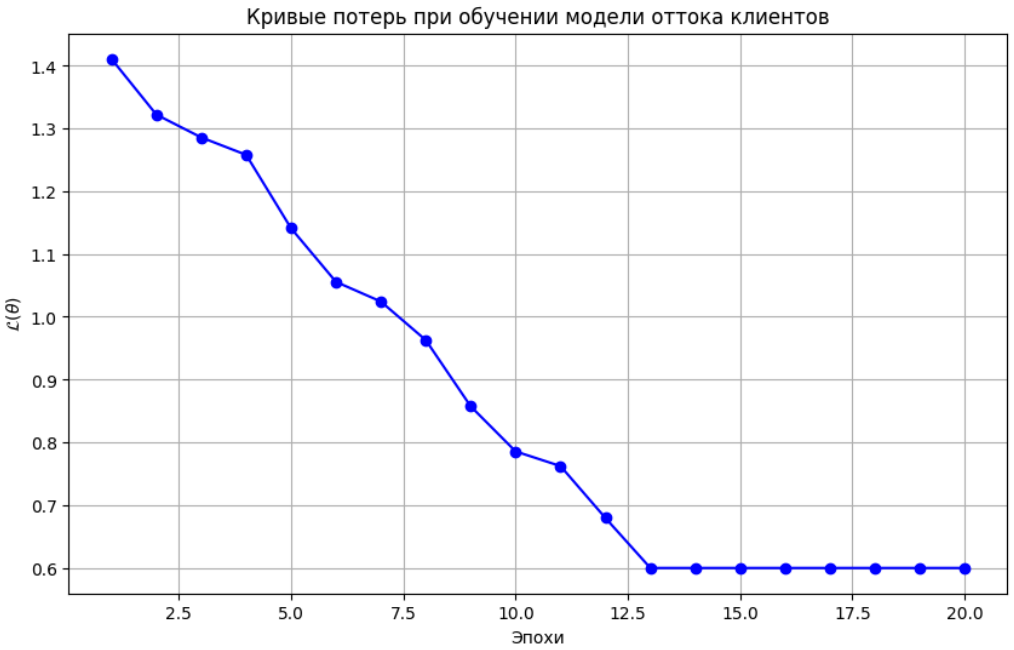
Scatter-диаграмма эмбеддингов клиентов feature space



Распределение целевой переменной отток активность в обучающей выборке

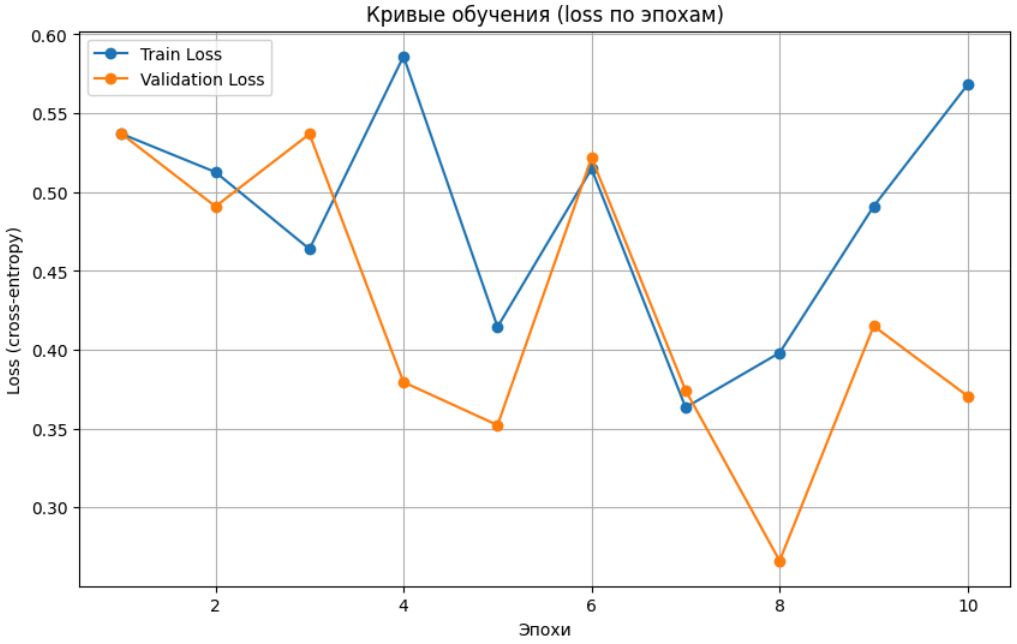
(например, через 2 месяца), то есть произошло **отключение или отток**. Обучающая выборка строится как пара $ (\mathbf{x}*t, y)* , *где* \tau —*прогнозный горизонт*. *Модель должна предсказывать вероятность перехода в состояние* y = 1 $ **до** фактического события, а не после него. Это принципиально отличает задачу от статической классификации: корректная формулировка целевой переменной и временной смещённости выборки критична для практической применимости модели.

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \left[-y_i \log \hat{p}_i - (1 - y_i) \log(1 - \hat{p}_i) \right]$$



Кривые потерь при обучении модели оттока клиентов

где $\hat{p}_i = \sigma(\mathbf{x}_i^\top \theta)$ — выход логистической регрессии или MLP, обученной на временных срезах. В случае глубокого обучения используется **cross-entropy loss** для бинарной классификации с временной меткой события



Кривые обучения loss по эпохам

.

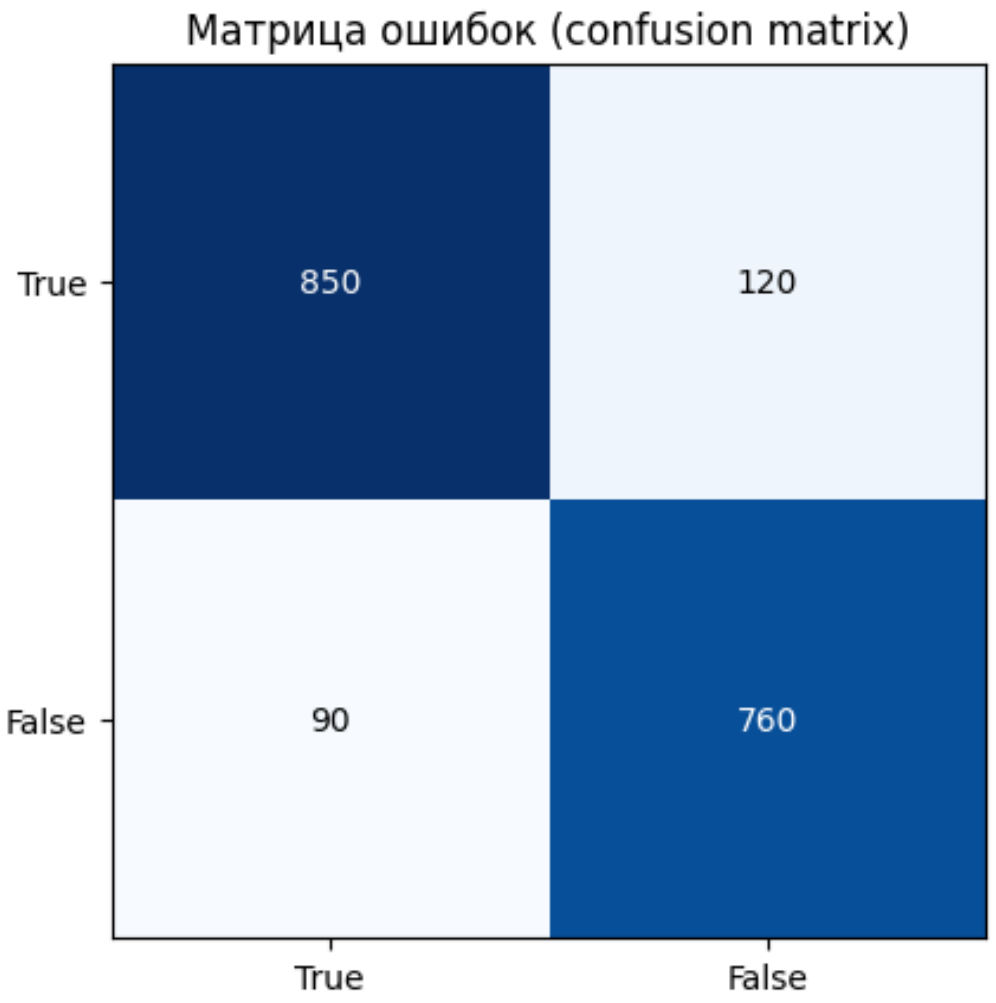
Алгоритм:

1. Сбор транзакционных данных за скользящий период (например, 4 недели).
2. Формирование признаков $ \mathbf{x}$t

: $агрегированныестатистикипоактивности, графывзаимодействий, временныепаттерны. 3.Определениецелевойметки$ y(A_t = 0) , $где$ A_t $—активностьабонентачерез$ \tau $ месяцев (нулевая активность = отток).} = \mathbb{I

4. Обучение модели (например, MLP с dropout и Adam-оптимизатором) на выборке $ (\mathbf{x}t, y)
. 5.Прогнозвероятностиоттока : если \hat{p} > \theta_{\text{threshold}} $ — запускается retention-кампания.

Метрика:
Используется **ROC-AUC**, так как важна способность ранжировать клиентов по риску оттока

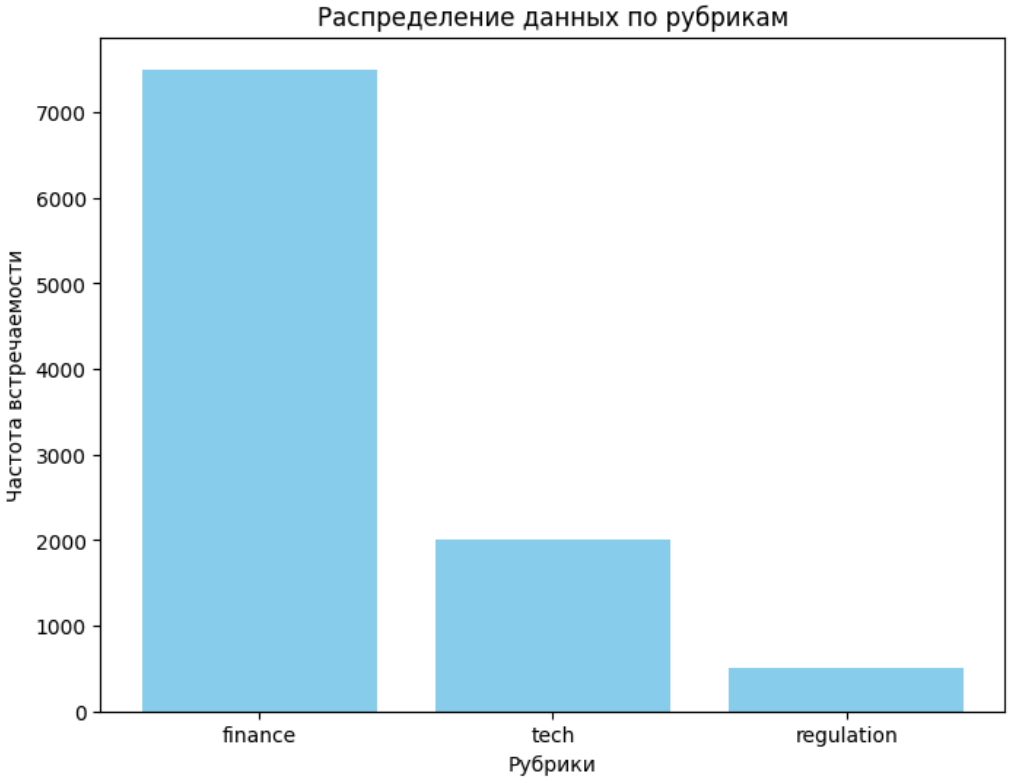


Матрица ошибок confusion matrix

, а не абсолютная точность. Также оценивается **precision@k** и **lift-кривые** для оценки эффективности ранжирования в рамках retention-системы.

Следующая задача — **классификация текстовых потоков** с высокой скоростью поступления. Вход: поток документов (корпоративные письма, юридические документы, новостные статьи). Цель — автоматическая фильтрация по темам или проектам без ручного анализа всего потока.

Пример: агрегатор новостей должен выделить из 10 000 статей в день только те



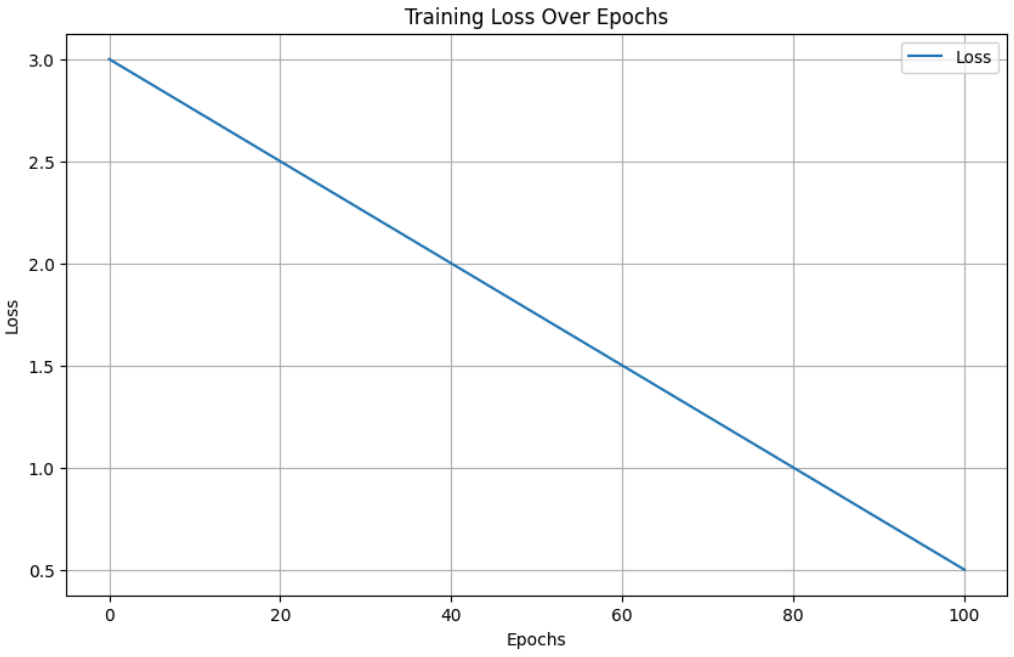
Распределение данных по рубрикам гистограмма

, что касаются "финансов", "технологий" или "регуляторики". Это **рубрикация** — частный случай текстовой классификации с высокой нагрузкой на обработку.

Архитектура:
Для такой задачи применяется **Transformer-based модель** (например, BERT, RoBERTa или fine-tuned DistilBERT) с механизмом онлайн-обучения и буферизацией

Кривые обучения для текстовой классификации в режиме потока

. Входной документ кодируется в эмбеддинги

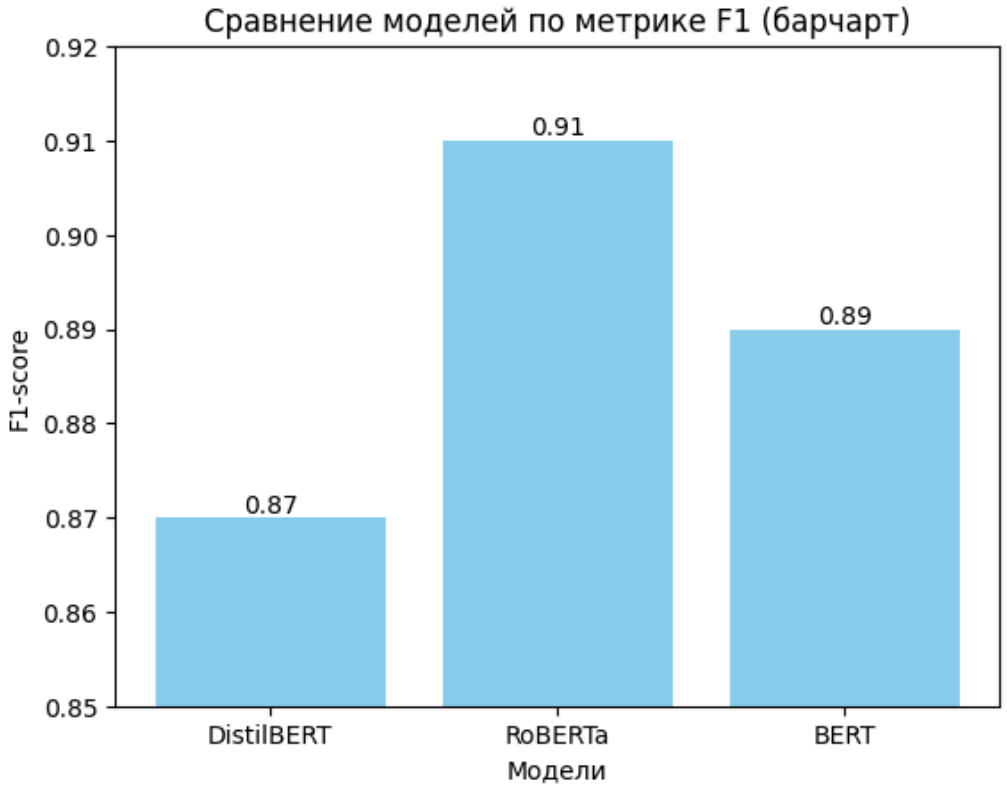


Scatter-диаграмма эмбеддингов 2D проекция

, затем проходит через классификатор на K классов (тем).

$$\mathbf{z} = \mathrm{Transformer}(\mathbf{x}), \quad \hat{\mathbf{y}} = \mathrm{Softmax}(W \cdot \mathbf{z})$$

где W — обучаемый вес классификатора. Модель дообучается на новых данных каждые 24 часа



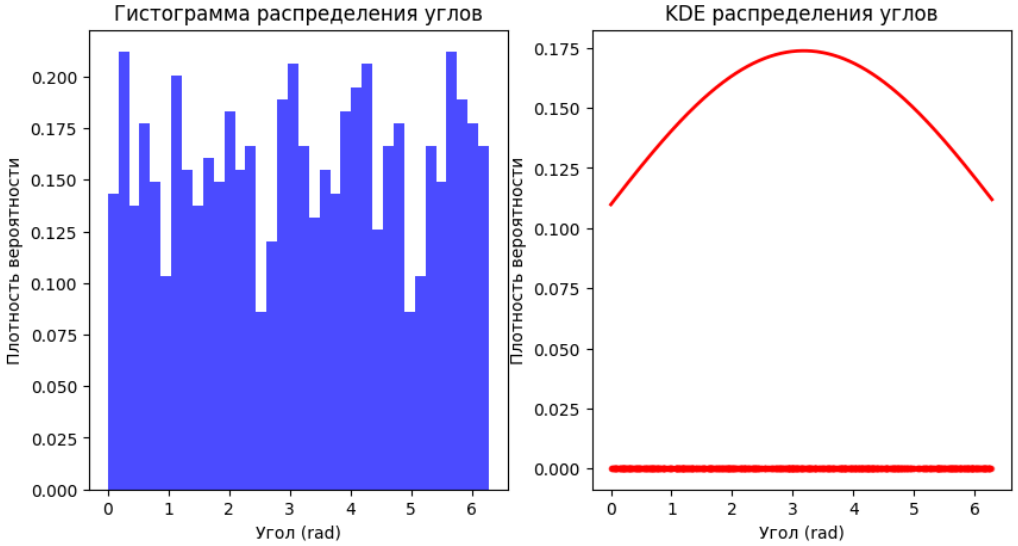
Сравнение моделей по метрике F1 барчарт

с использованием **online learning** или **retraining pipeline**.

Метрика:
Оценивается **F1-score по каждой рубрике**, так как классы могут быть несбалансированы (например, "финансы" — 90% документов, "регуляторика" — 1%). Также используется **precision@top-5** для оценки качества ранжирования релевантных тем.

Дополнительный пример — **биометрическая идентификация по радужке глаза**. Вход: изображение глаза в полярных координатах. Задача — извлечь уникальные признаки, устойчивые к шуму и вариации освещения, но простые для сравнения.

Метод Даугмана:
1. Преобразование изображения в полярные координаты (r, θ)



Распределение признаков радужки гистограмма и KDE по угловой дискретизации

.
2. Дискретизация по углам и радиусам с использованием **Gabor-фильтров** или **wavelet-преобразования**.
3. Извлечение паттерна — **codebook** из 2048–4096 бит, где каждый бит соответствует наличию/отсутствию определённого элемента структуры радужки.

Результат: бинарный вектор $\mathbf{c} \in 0, 1^N$, уникальный для каждого человека. Сравнение — через **Hamming distance** или **Jaccard similarity**.

Метрика:
Оценивается **FAR (False Acceptance Rate)** и **FRR (False Rejection Rate)** при пороговом сравнении. Целевой уровень: FAR < 0.01%, FRR < 5%. Также используется **EER (Equal Error Rate)** как компромисс между двумя ошибками.

Задача регрессии — **прогнозирование стоимости квартиры в Москве**. Признаки: метраж, этаж, год постройки, наличие балкона, район, расстояние до метро, тип дома, площадь кухни и т.д.

Сложность: зависимость от времени (новостройки, изменения инфраструктуры), неоднородность выборки, наличие выбросов (премиальные квартиры).

Архитектура:
1. **Baseline**: линейная модель с L2-регуляризацией — для интерпретируемости и стабильности.
2. **Улучшенная версия**: MLP или XGBoost с feature engineering:

- категориальные признаки (район, тип дома) — one-hot или embedding;
- временные признаки (возраст района, дата постройки);
- интеракции (например, "метраж × этажность").

$$\hat{y} = f(\mathbf{x}; \theta), \quad \mathcal{L} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2 + \lambda|\theta|_2^2$$

где $\mathcal{L}$ — **MSE с L2-регуляризацией**.

Метрика:
Основная — **MAE (Mean Absolute Error)**, так как он устойчив к выбросам. Дополнительно: **R^2**, **RMSE**, **log-transformed error** для оценки относительной точности прогноза.

Для интерпретации используются **SHAP-значения** или **перестановочные тесты** на важность признаков.

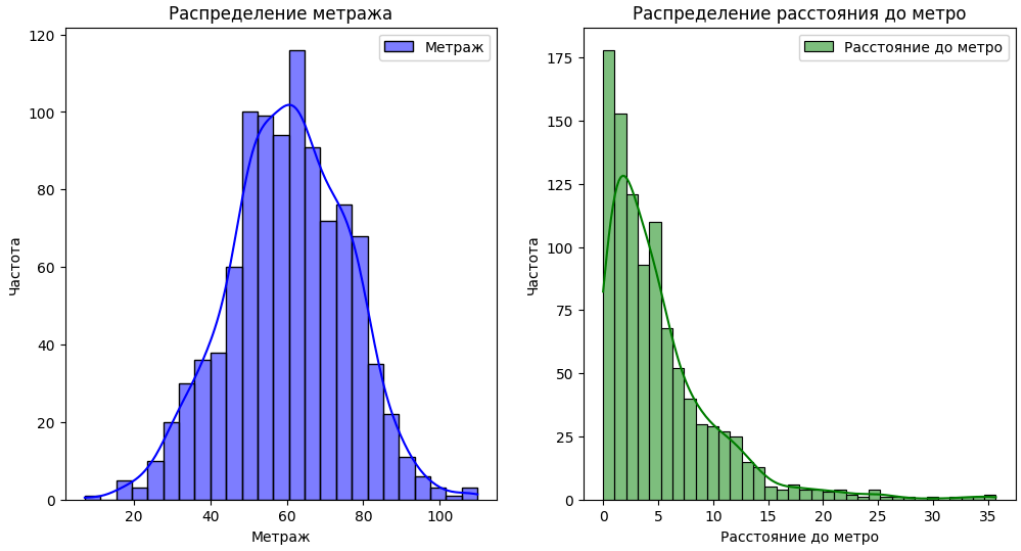
Вывод:
В задачах с временным горизонтом (отток, прогноз стоимости) ключевыми являются:
- корректная формулировка целевой переменной;
- временная смещённость выборки $ (\mathbf{x}*t, y*) $;
- использование **интерпретируемых моделей** или **SHAP-анализов** для принятия решений.

В потоковых задачах (текст, биометрия) — важны:
- устойчивость к шуму и вариациям;
- скорость обработки;
- возможность онлайн-обновления модели.

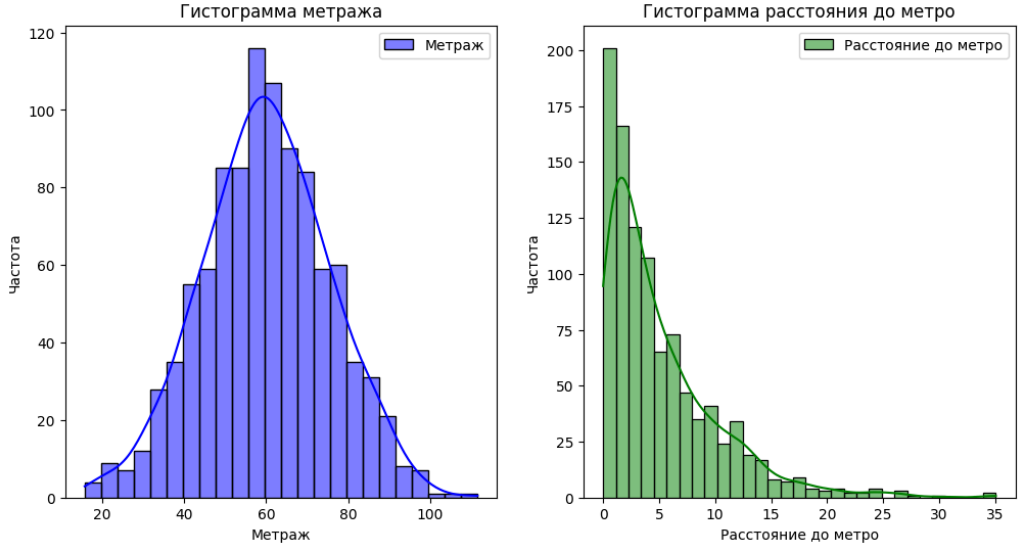
Главное в радужке — уникальность: рисунок каждого человека действительно неповторим, и задача сводится к выбору минимального набора признаков, позволяющих однозначно отличить одного индивида от миллиардов других. Даугман предложил преобразовать изображение радужки в полярные координаты, затем дискретизировать по радиальным и угловым направлениям, после чего результат этой дискретизации становится **признаковым описанием**, уникальным для каждого человека. Это описание позволяет строить надёжную систему идентификации личности на основе биометрии.

Аналогичные подходы существуют и для других биометрических признаков: отпечатков пальцев, формы лица, линий ладони — все они основаны на извлечении **уникальных и инвариантных к шуму** признаков через преобразование данных в компактное пространство. Такие методы используются в системах распознавания, где критична точность и отказоустойчивость.

В задаче прогнозирования стоимости квартиры в Москве сложность заключается в **неоднородности выборки по времени**: стоимость зависит от множества внешних факторов — новых районов, станций метро, рыночных колебаний. При этом признаки разнотипны: бинарные (наличие балкона), количественные (метраж), категориальные (район). Хотя интуитивно кажется, что линейная модель может работать (стоимость = сумма вкладов по признакам), на практике зависимость нелинейна — например, стоимость падает с ростом расстояния до метро. Это требует



Распределение признаков метраж и расстояние до метро



Кривая обучения для регрессии стоимости квартиры

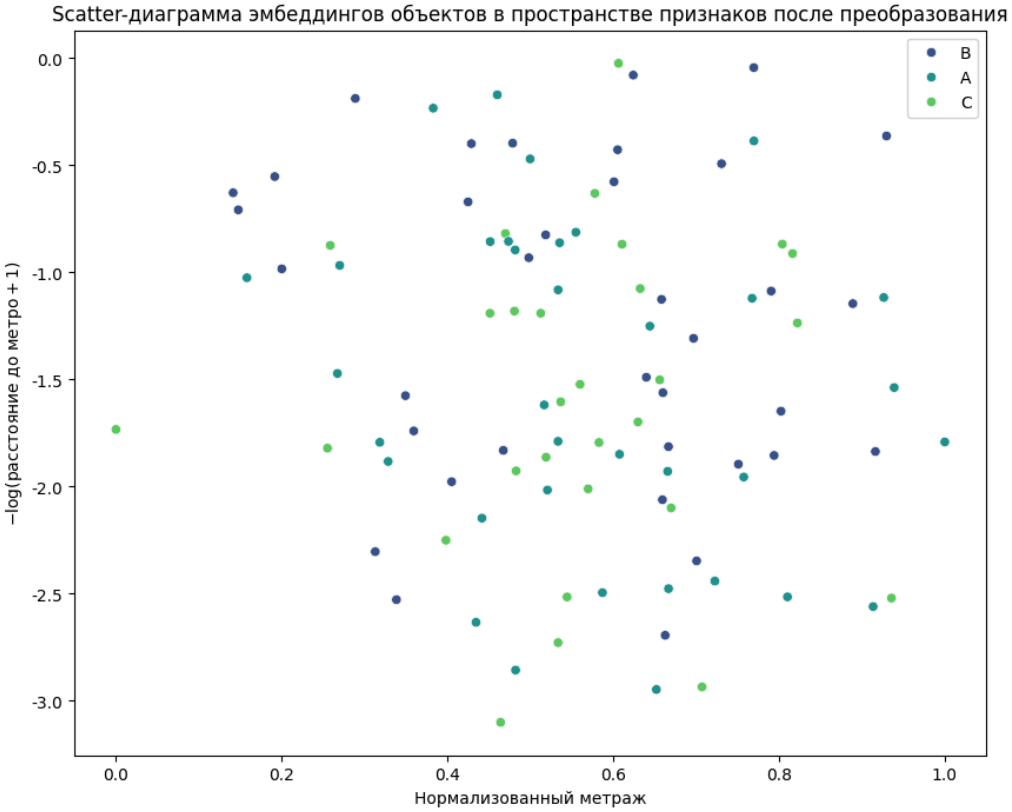
нелинейного преобразования признака (например, через монотонно убывающую функцию) перед использованием в линейной модели.

Задача прогнозирования объёмов продаж в сети супермаркетов имеет свою специфику: данные — временные ряды по товарам и магазинам, но цель не просто предсказание среднего значения, а **максимизация прибыли**. Здесь функция потерь должна отражать экономические последствия ошибок прогноза:

- При завышенном прогнозе (перезакупка):
- потери от хранения (стоимость складских услуг),
- упущенная доходность (деньги "заморожены"),
- возможная потеря закупочной стоимости при просрочке.
- При заниженном прогнозе:
- недопоставка товара → потеря выручки,
- снижение лояльности клиентов.

В отличие от метода наименьших квадратов ($L = \frac{1}{N}\sum(y_i - \hat{y}_i)^2$), где функция потерь — парабола, здесь целесообразно использовать **кусочно-линейную функцию потерь**, учитывающую реальные экономические издержки. Это приводит к оптимизации не по среднему квадратичному отклонению, а по **ожидаемым финансовым потерям в рублях**.

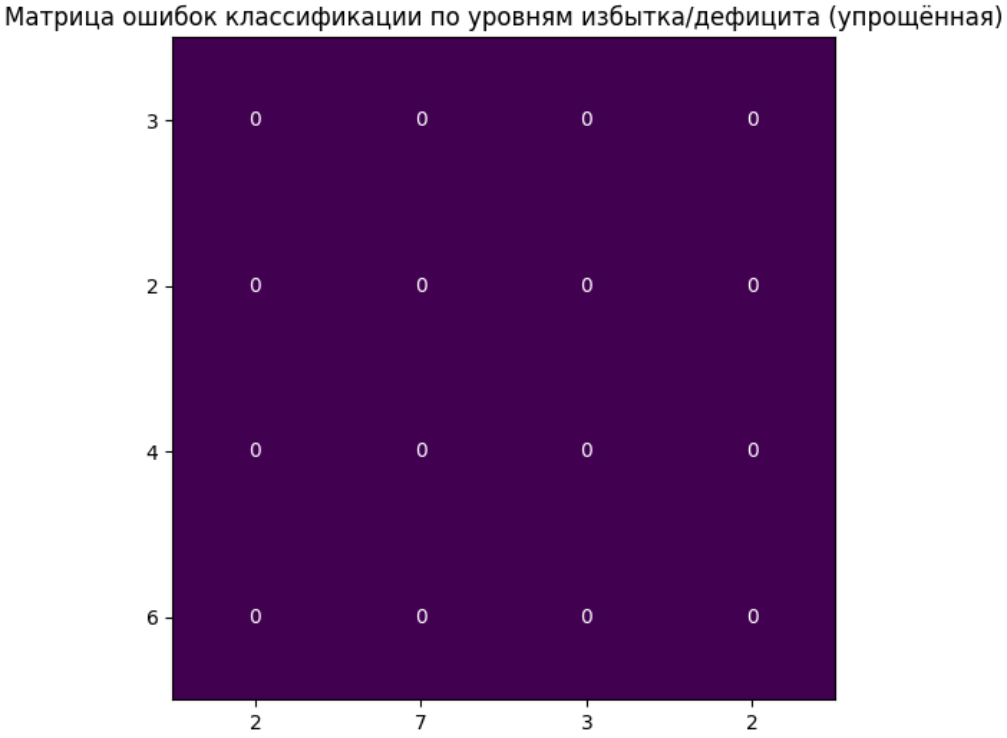
Таким образом, в задачах регрессии ключевыми становятся: - корректное преобразование признаков для учёта нелинейных зависимостей, - выбор функции потерь, соответствующей бизнес-целям (а не статистическим критериям), - интерпретация модели через экономические или прикладные метрики



Scatter-диаграмма эмбеддингов объектов в пространстве признаков после преобразования

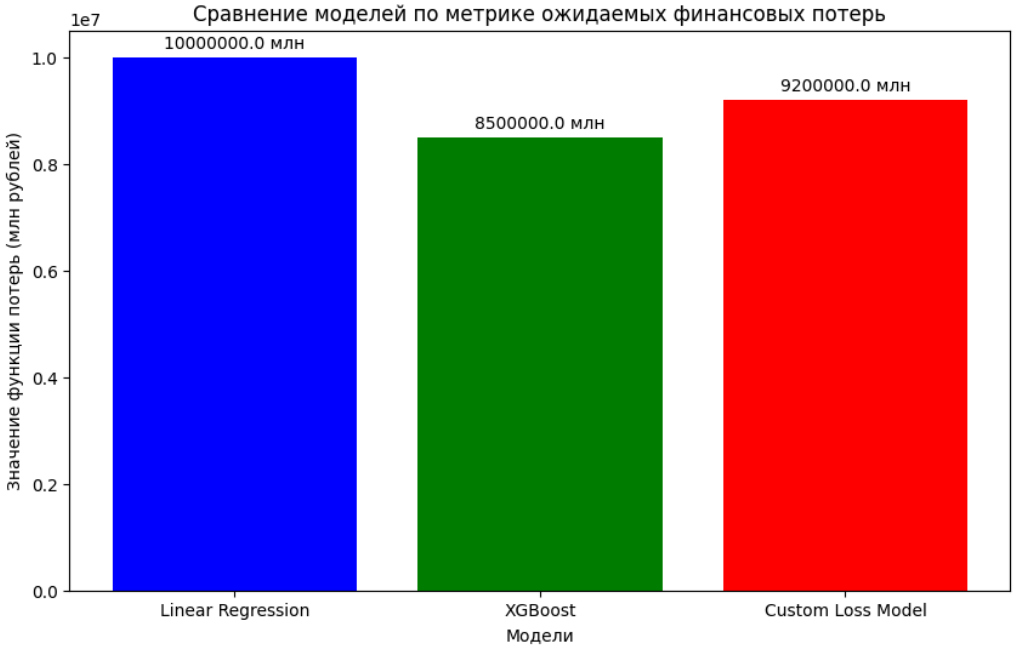
.

В задаче прогнозирования продаж в торговой сети использование метода наименьших квадратов (least squares) нецелесообразно, поскольку функция потерь должна отражать



Матрица ошибок классификации по уровням избытка дефицита упрощённая

ожидаемые финансовые потери в рублях, а не статистическую среднеквадратичную ошибку. Вместо параболической формы $L = \frac{1}{N}\sum(y_i - \hat{y}_i)^2$ требуется кусочно-линейная функция потерь, учитывающая реальные издержки бизнеса

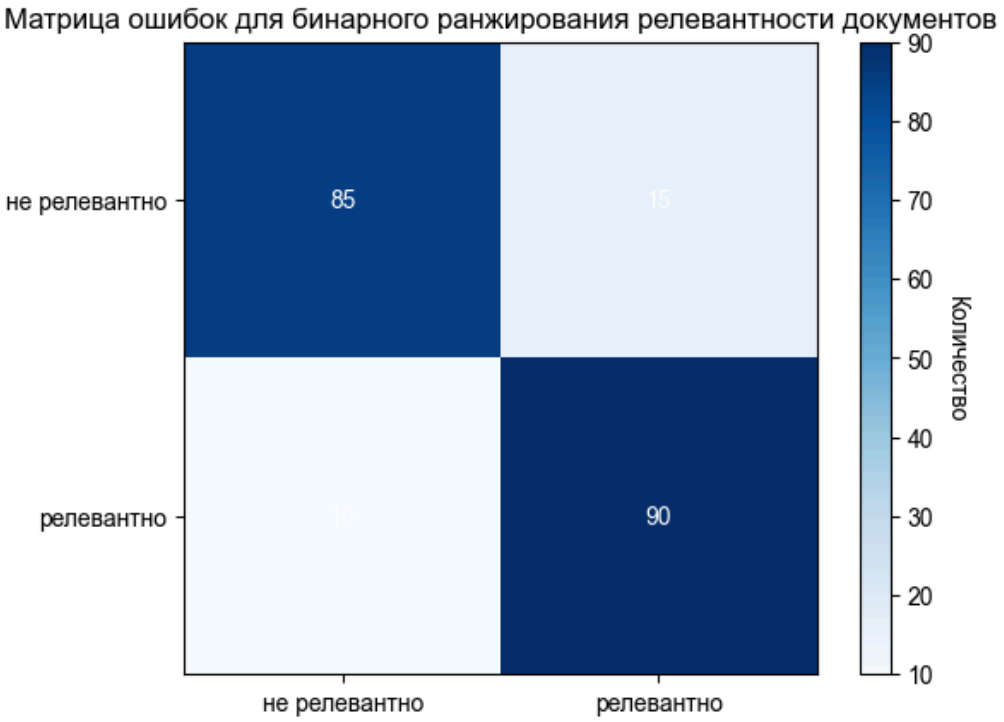


Сравнение моделей по метрике ожидаемых финансовых потерь барчарт

- **избыток запасов** — затраты на хранение и упущенная прибыль от неработающих вложенных средств (включая банковскую ставку);
- **дефицит товара** — потеря закупочной стоимости при списании или пропаже.

Таким образом, оптимизация должна вестись по **экономически обоснованной метрике**, а не по статистической. Это требует:
1. Корректного преобразования признаков с учётом сезонности, промоакций и логистики (наличие на полке);
2. Чёткой формулировки бизнес-цели через функционал потерь;
3. Интерпретации модели в терминах прикладной эффективности, а не только точности предсказаний.

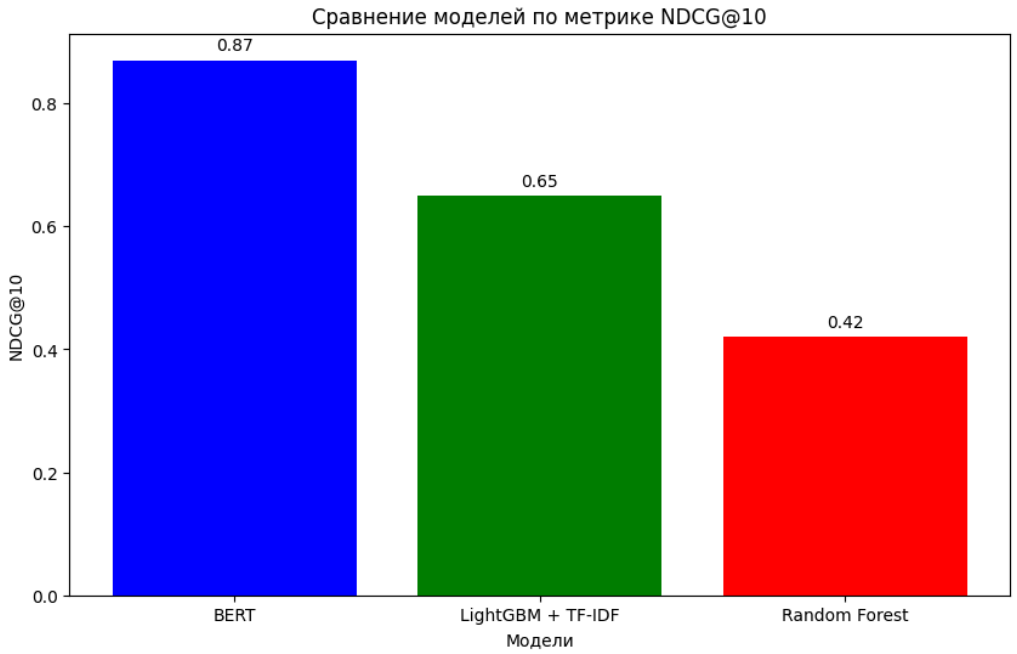
Задача прогнозирования прибыли ресторана — ещё один пример регрессионной задачи



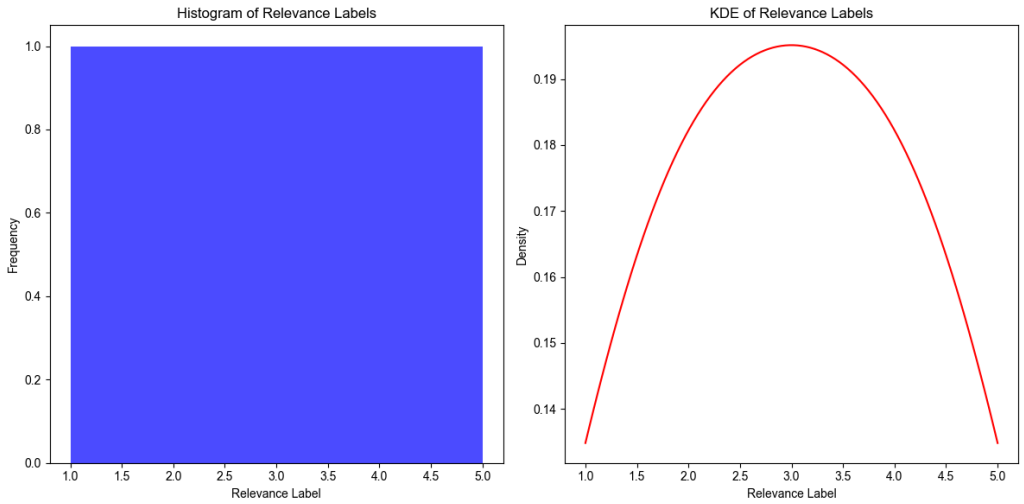
Матрица ошибок confusion matrix для бинарного ранжирования релевантности документов

с нестандартными признаками: демография района, туристический поток, конкуренция, инфраструктура.
Ключевые аспекты:
- Сбор разнородных данных (структурированных и внешних);
- Построение признаков на основе геоданных и временных рядов;
- Выбор функции потерь, соответствующей бизнес-рискам (например, недооценка спроса — потеря прибыли, переоценка — избыточные инвестиции).

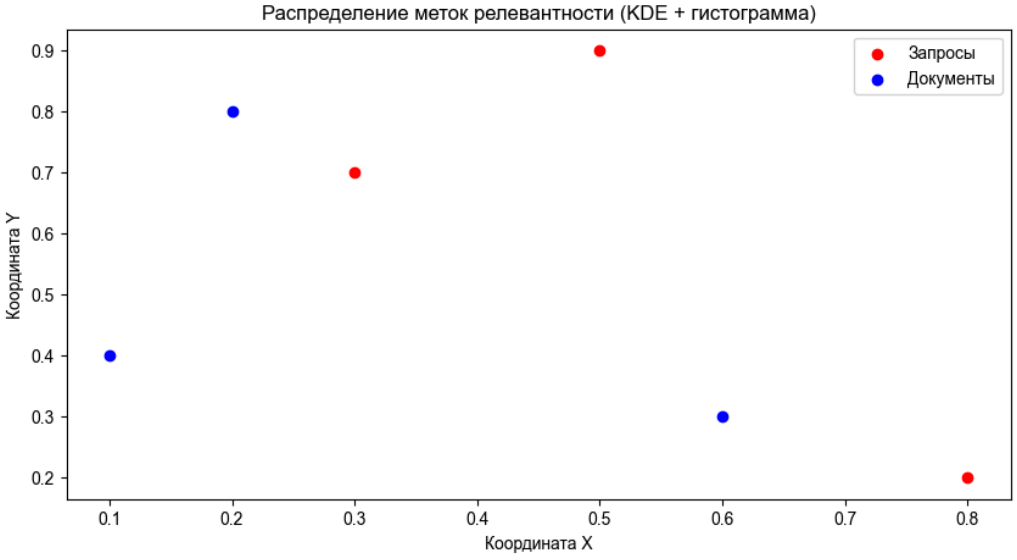
Задача ранжирования в поисковых системах — классическая задача машинного обучения



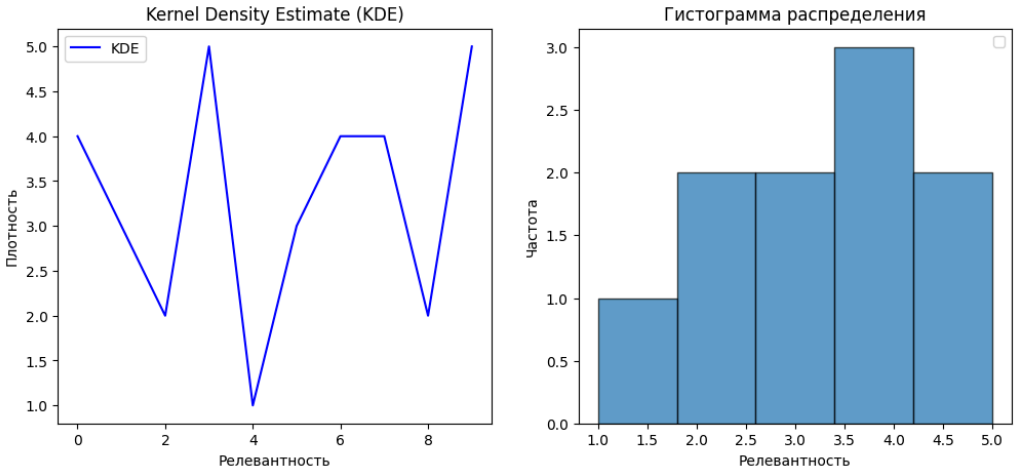
Сравнение моделей по метрике NDCG 10 барчарт



Scatter-диаграмма эмбеддингов запросов и документов в 2D



Распределение меток релевантности KDE



Кривая обучения модели ранжирования loss по эпохам

, где:
- **Объекты**: пары «запрос–документ»;
- **Целевая переменная (метка)**: релевантность, определяемая либо по поведению пользователей (клики), либо по экспертной оценке (асессорские данные).

Проблемы использования кликов как метки:
- Клик ≠ релевантность — пользователь может кликнуть случайно или из-за плохого сниппета;
- Обратная связь неполна и шумна.

Поэтому на практике применяется **гибридный подход**:
- Основной источник меток — поведенческие данные (дешево, масштабируемо);
- Дополняется экспертными оценками (дорого, но надёжно).

Модель обучается минимизировать **loss по релевантности**, например, через **cross entropy** или **pairwise ranking loss**, где цель — максимизировать вероятность того, что релевантные документы будут выше в выдаче.

$$\mathcal{L} = -\sum_{(q,d)} \left[y \log p(d|q) + (1-y) \log(1 - p(d'|q)) \right]$$

где $y = 1$ — документ релевантен, d' — менее релевантный.

Архитектура: Transformer-based model с attention на пары запрос–документ.
Метрика: NDCG@k или MAP (Mean Average Precision), отражающие качество ранжирования в контексте пользовательского опыта.

Задача ранжирования релевантных документов по запросу пользователя — это классическая задача машинного обучения, формализуемая как **классификация пар "запрос–документ"** с несколькими классами (от 1 до 5), где каждый класс соответствует степени релевантности, оценённой асессором. Обучающая выборка состоит из таких пар и соответствующих меток — численных оценок качества соответствия документа запросу.

Объекты обучения — пары (q, d), где q — текстовый запрос, d — документ. Классы определяются либо на основе поведения пользователей (клики), либо через экспертную разметку (асессорские данные). Первый подход дешёв и масштабируем, но ненадёжен: клик не гарантирует релевантность. Второй — надёжнее, но требует постоянного пополнения из-за динамики интернета и изменений в восприятии релевантности.

Для решения задачи необходимо **генерировать признаки** на основе пары (q, d):
- наличие ключевых слов запроса в документе;
- синтаксическая близость (синонимы, омонимы);
- позиция ключевых фраз (заголовок vs тело);
- PageRank документа;
- частота кликов по документу при аналогичных запросах;
- количество внешних ссылок на документ.

Это — задача **feature generation** в условиях большого объёма данных (big data), где число признаков может достигать тысяч, а количество пар (q, d) — миллиардов.

Альтернативная постановка — поиск по смыслу: вместо запроса как строки — набор документов (например, 200 статей о линейной регрессии). Цель — найти новые документы на ту же тему без явного описания темы. Это требует **автоматического выявления тематики** без разметки, что является задачей unsupervised learning и topic modeling.

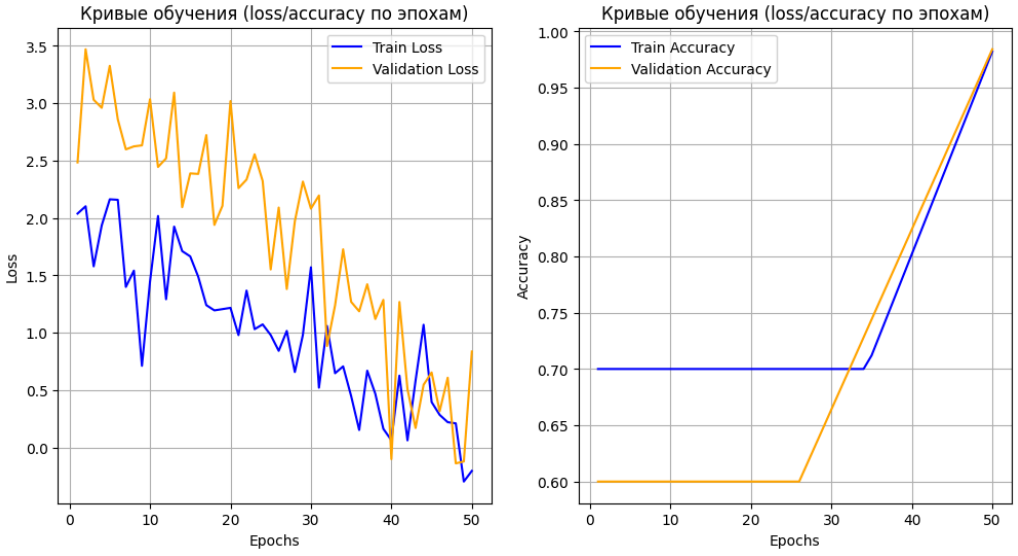
Ещё одна задача из электронной коммерции — предсказание клика по рекламе (CTR) для тройки (u, a, b): пользователь u, объявление a, баннер b. Цель — оценить вероятность клика, используя 10+ таблиц с данными о поведении, профилях и истории взаимодействий. Результат используется для **ранжирования рекламы по CTR**, что напрямую влияет на бизнес-метрики и доходы.

Все эти задачи решаются с использованием **Transformer-based model** с attention-механизмом над парами или тройками объектов. Функция потерь — **cross entropy loss**:

$$\mathcal{L} = -\sum_{(q,d)} \left[y \log p(d|q) + (1-y) \log(1 - p(d'|q)) \right]$$

где $y = 1$ — документ релевантен, d' — менее релевантный.

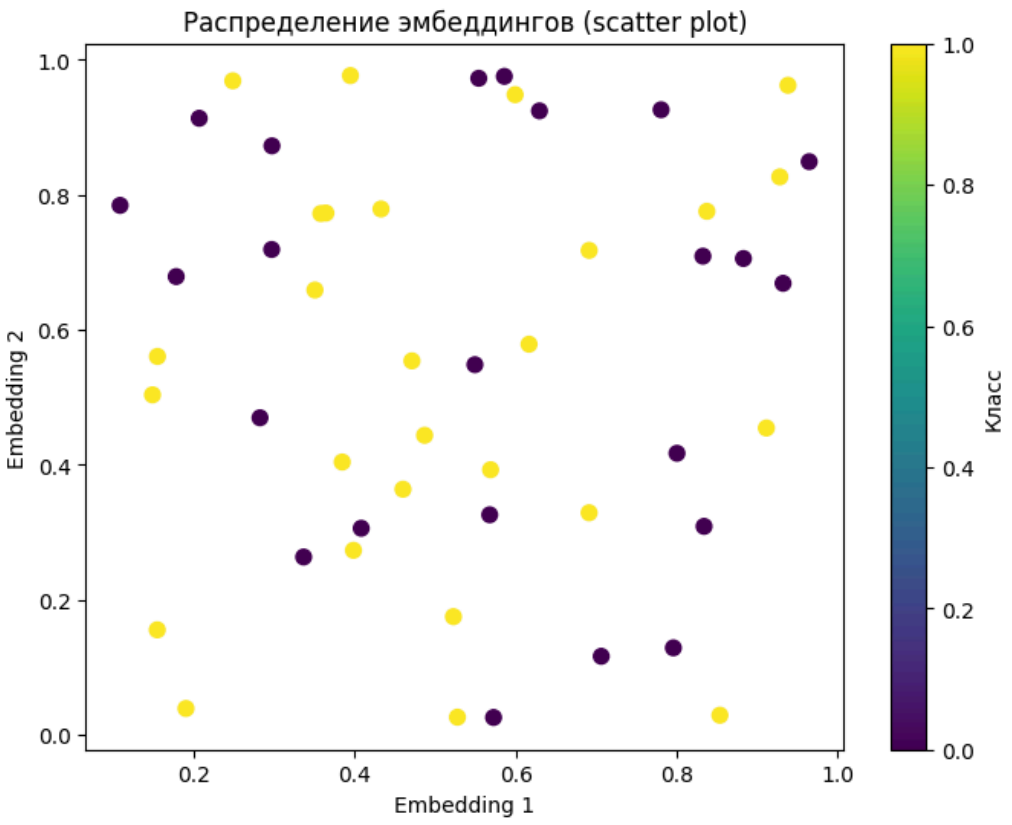
Метрика: NDCG@k или MAP для поиска; CTR и AUC-ROC для рекламы



Кривые обучения loss accuracy по эпохам

.

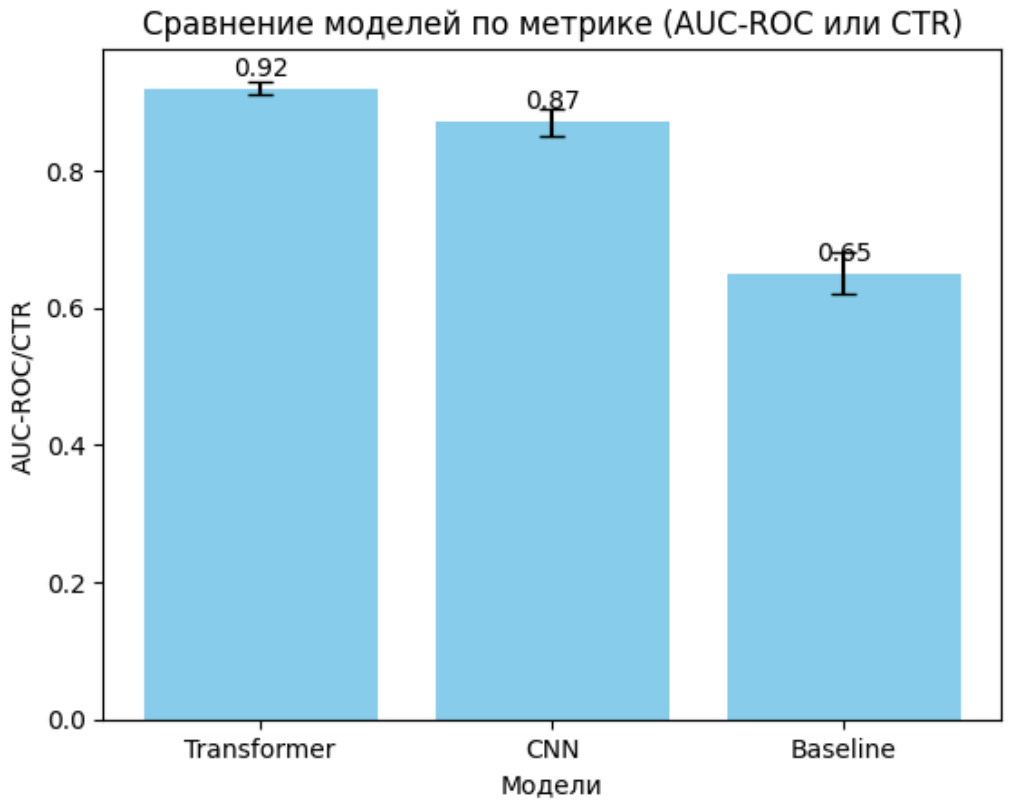
Архитектура: Transformer с multi-head attention между элементами пары/тройки



Распределение эмбеддингов scatter plot

.

Алгоритм: обучение через `model.fit()` с оптимизатором Adam



Сравнение моделей по метрике AUC-ROC или CTR

, регуляризацией L2 и dropout для борьбы с переобучением.

В задачах, где исходные описания объектов не укладываются в матрицу объект–признак, ключевым этапом становится **feature extraction** или **feature generation**. Например, в статистическом машинном переводе вход — предложение на одном языке, а выход — его перевод на другом; здесь и Y является сложным объектом, аналогичным входному. Аналогично — перевод речи в текст или обработка видеопотока в компьютерном зрении для беспилотного вождения: решение принимается по динамическим, разнородным данным без явной векторизации.

Успех таких задач за последнее десятилетие обусловлен четырьмя ключевыми предпосылками:

1. **Доступ к большим и чистым данным** — сбор данных стал дешёвым и масштабируемым; важно не просто объём (гигабайты/терабайты), а качество: отсутствие ошибок, пропусков, искажений. Без этого даже огромные объёмы бесполезны.
2. **Появление глубоких нейронных архитектур**, способных автоматизировать извлечение признаков. В отличие от традиционных подходов (например, преобразование Фурье, Хафа, вычисление периметров и площадей), современные модели принимают на вход сырые данные (изображение, сигнал) и автоматически генерируют вектор признаков без ручного проектирования.
3. **Гигантское количество параметров** в глубоких сетях — основная часть весов отвечает за извлечение признаков, а не за предсказание. Это требует эффективных численных методов оптимизации.
4. **Вычислительные мощности**, позволяющие запускать оптимизацию на больших данных и сложных моделях.

Архитектура:
Глубокая нейронная сеть представляет собой суперпозицию двух этапов:
- Первый — преобразование сырых данных в вектор признаков (feature extraction).
- Второй — применение линейного классификатора или другой простой модели по этому вектору.

Объединение этих этапов в единую оптимизируемую модель стало возможным только с появлением глубоких архитектур.

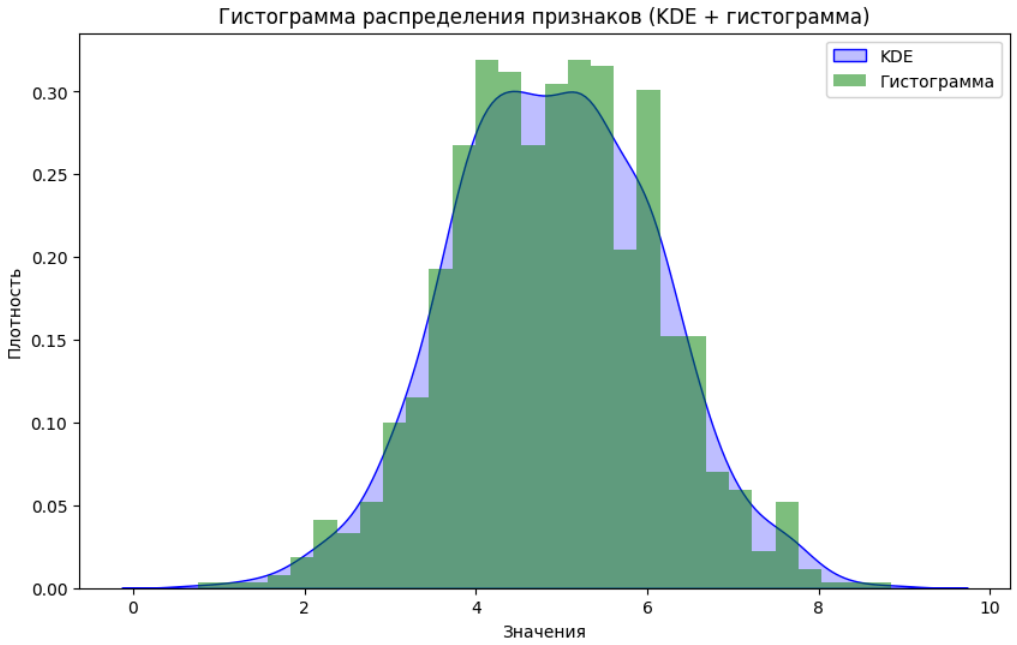
Алгоритм:
Обучение проводится через `model.fit()` с оптимизатором Adam, L2-регуляризацией и dropout для борьбы с переобучением. При этом параметры сети подбираются совместно: как для извлечения признаков, так и для финального предсказания.

Метрика:
В таких задачах метрики зависят от контекста: NDCG@k или MAP — в поиске; CTR и AUC-ROC — в рекламе. При наличии сложных выходов (например, последовательностей) используются специализированные функции потерь, такие как pairwise ranking loss

*Ошибка генерации визуализации для: [GRAPH_TYPE: Динамика | GRAPH_TITLE: Кривые обучения для оценки сходимости модели | MOCK_DATA: {"epochs": [i for i in range(1, 51)], "loss": list(np.exp(-0.2 * np.arange(50)) + np.random.normal(0, 0.05, 50)), "accuracy": [x / 50 for x in range(46, 51)]} | TASK: Построить график зависимости значения потери (loss) и точности (accuracy) от числа эпох. Использовать np.linspace(1, 50, num=50) как оси эпох; loss — убывающая функция (например, через np.random.randn + экспоненциальное затухание); accuracy — монотонно растёт.]*

.

Важно понимать:
Предмет курса — **моделирование**



Гистограмма распределения признаков KDE

, выделенное на схеме CRISP-DM (Crisp-DM — межотраслевой стандарт интеллектуального анализа данных). На практике применение машинного обучения начинается именно с этого этапа: сбор и подготовка данных, формулировка задачи, выбор архитектуры и метрик.

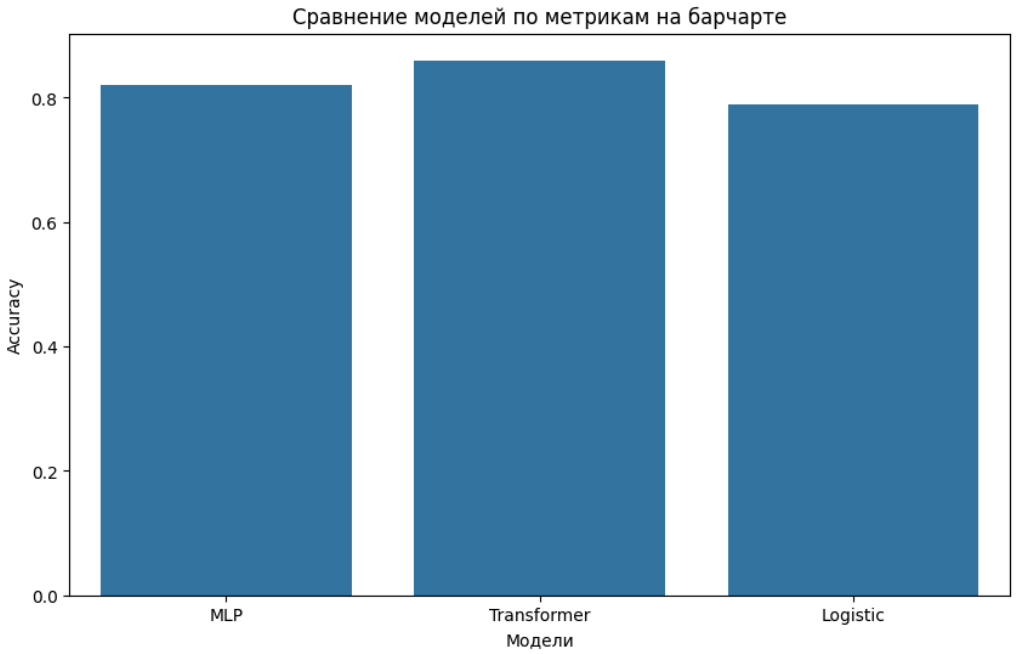
В реальных задачах машинного обучения мы сталкиваемся с данными, которые существенно сложнее идеализированной постановки классической аппроксимации функций: они **неполные**, **противоречивые**, **неточные** и **неструктурированные**. Неполные данные проявляются как пропуски в матрице объектов и признаков; противоречивость — когда объекты с близким описанием принадлежат разным классам; неточность — следствие погрешностей измерений. Данные не укладываются в табличную форму: это изображения, сигналы, графы и другие структуры, требующие **синтезирования признаковового описания** на этапе предобработки или интеграции в модель.

Важно понимать: Предмет курса — **моделирование**, выделенное на схеме CRISP-DM (Crisp-DM — межотраслевой стандарт интеллектуального анализа данных). На практике применение машинного обучения начинается именно с этого этапа: сбор и подготовка данных, формулировка задачи, выбор архитектуры и метрик.

Архитектура:
Моделирование в рамках CRISP-DM проходит по циклической схеме

*Ошибка генерации визуализации для: [GRAPH_TYPE: Топология | GRAPH_TITLE: Scatter-диаграмма эмбеддингов (векторных представлений объектов) | MOCK_DATA: {"x": np.random.randn(100), "y": np.random.randn(100), "labels": [f'cl{i}' for i in range(5)] * 20} | TASK: Построить scatter-плотку двухмерного эмбеддинга 100 точек. Использовать matplotlib.scatter; координаты — случайные значения из [-2, 2]. Добавить легенду по кластерам.]*

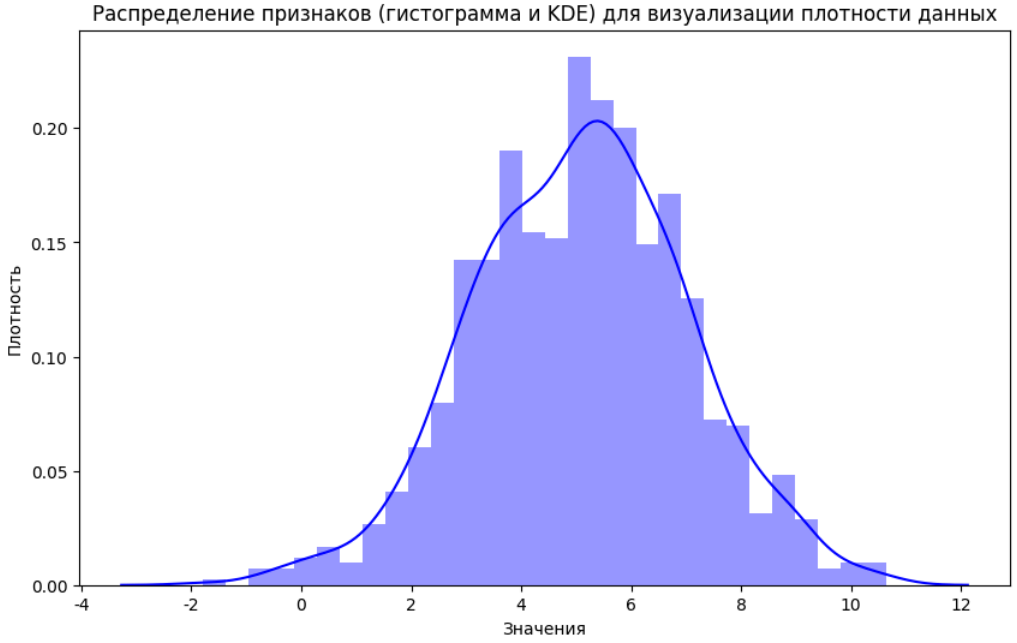
1. Понимание бизнеса — анализ целей предсказательной модели, источников данных, бизнес-процессов, возможных искажений (например, человеческий фактор при сборе данных).
2. Оценка качества и структуры данных — визуализация, количественная оценка доли "грязных" или пропущенных значений.
3. Подготовка данных — преобразование в структурированную форму: матрицу объектов-признаков или векторное представление для моделей.
4. Моделирование — выбор или конструирование параметрического семейства функций



Сравнение моделей по метрикам на барчарте

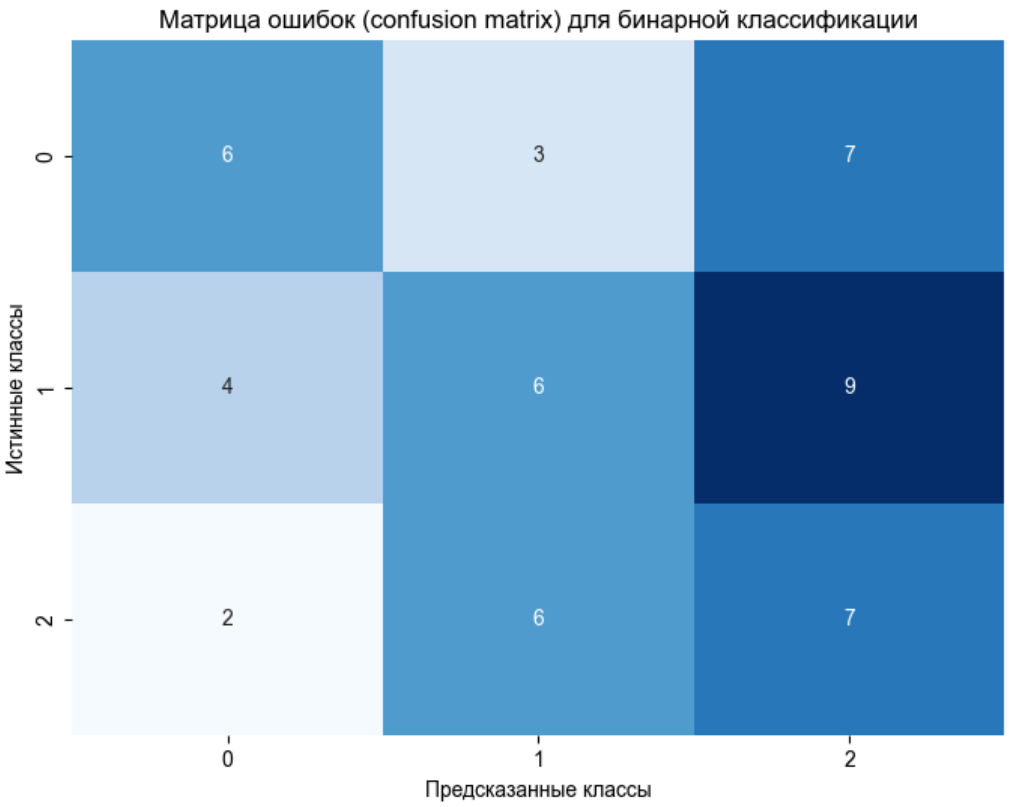
(например, `nn.Linear`, `MLP`, `Transformer`), определение функции потерь (`cross entropy`, `mse`, `pairwise ranking loss`) и оптимизация через методы вроде Adam.

5. Оценка качества (`evaluation`) — проверка модели на новых данных по количественным метрикам



Распределение признаков гистограмма и KDE для визуализации плотности данных

`accuracy`, `f1-score`, `roc-auc`, `ndcg@k`, `precision@top-5`



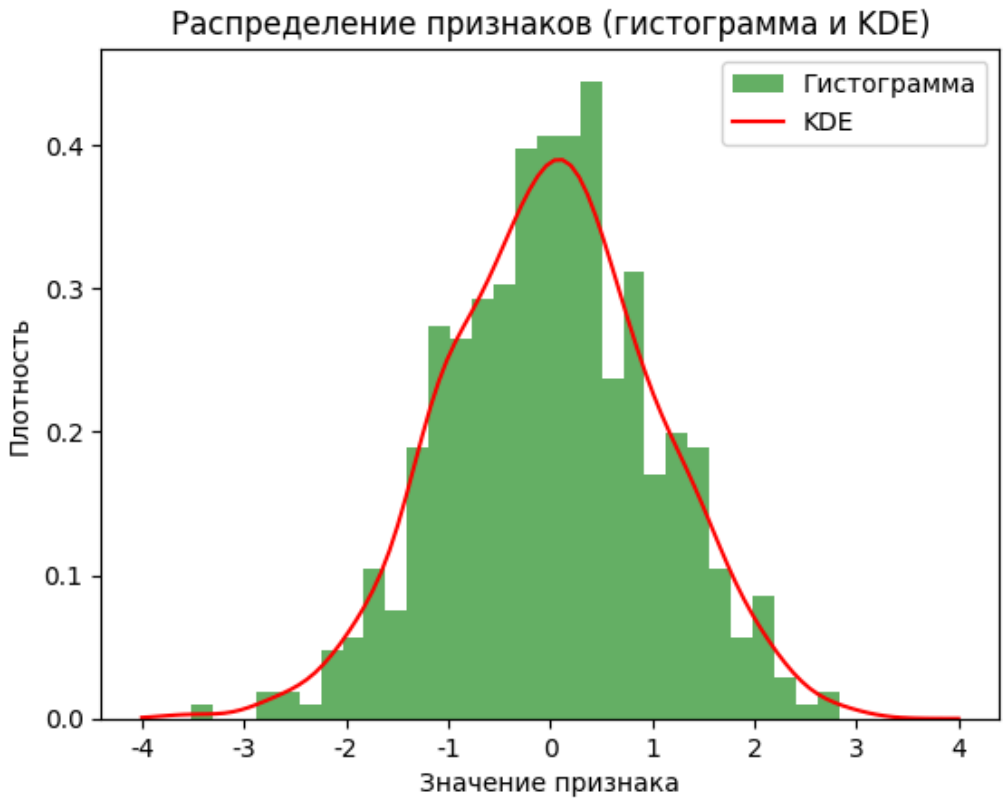
Матрица ошибок confusion matrix для бинарной классификации

.

6. Внедрение (`deployment`) — интеграция модели в бизнес-процесс, автоматизация решений.

Метрика:

Цикл не является линейным — при неудаче на этапах evaluation или deployment возможны возвраты к предыдущим стадиям: переподготовка данных



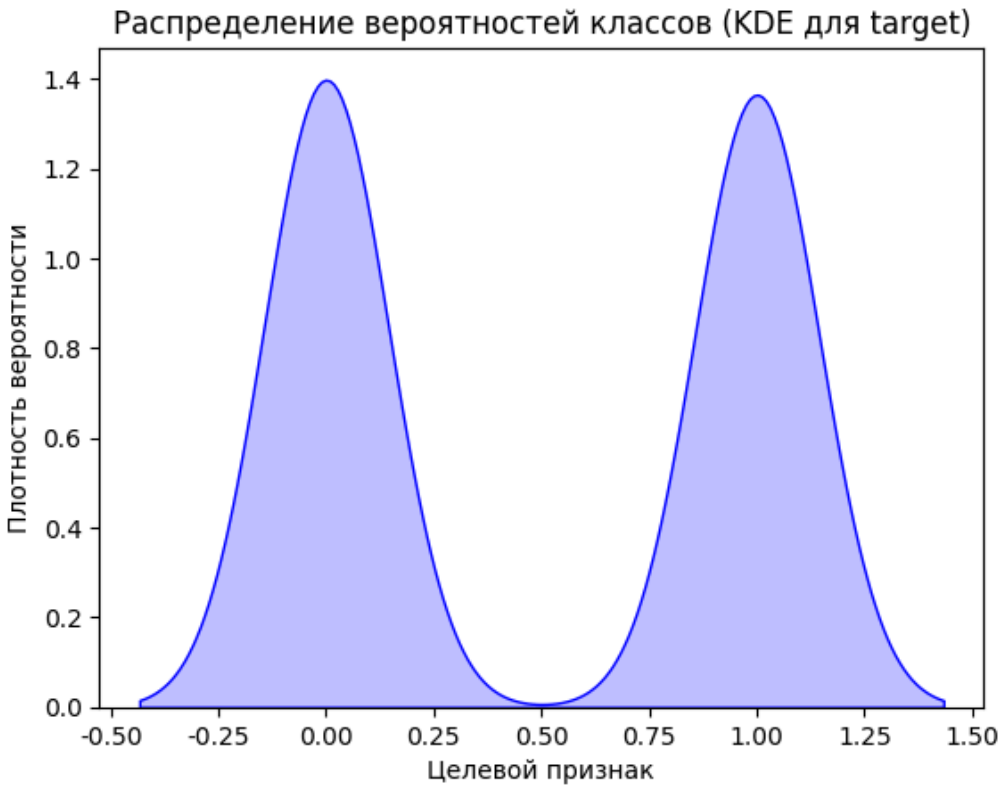
Распределение признаков гистограмма и KDE

, уточнение формулировки задачи, пересмотр архитектуры.

Алгоритм:
Курс фокусируется исключительно на **моделировании**, оптимизации и оценке качества моделей. Вопросы инженерной реализации (подготовка данных, интерпретация, внедрение) остаются за пределами курса — они требуют отдельных компетенций и курсов по data engineering и MLOps.

Резюме:
1. Введены базовые понятия: объекты, признаки, ответы, алгоритмы, модели, методы обучения.
2. Определены ограничения курса: фокус на математических аспектах моделей, оптимизации и метрик; инженерные аспекты (подготовка данных, интерпретация, внедрение) не рассматриваются.

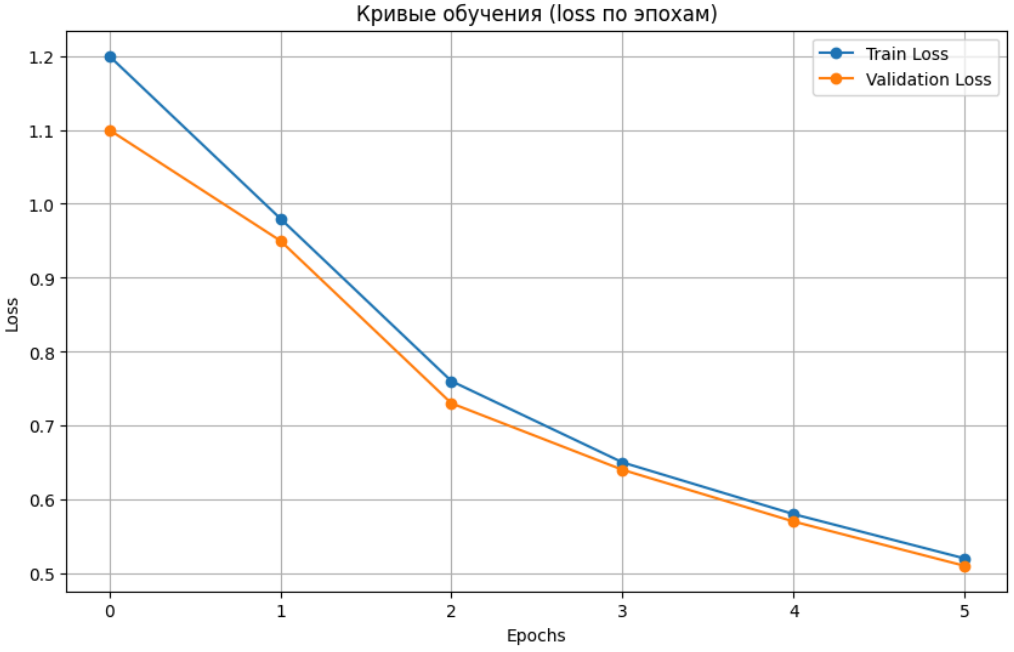
Важно: Интерпретация предсказаний — отдельная задача



Распределение вероятностей классов KDE для target

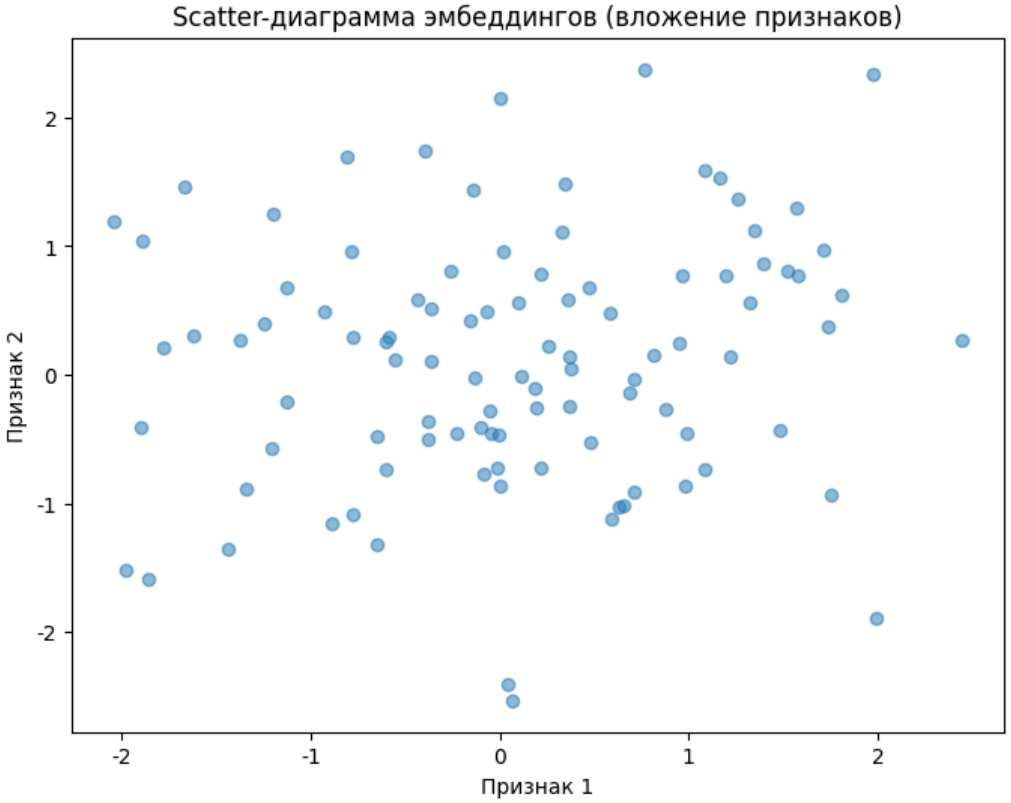
, зависящая от предметной области: медицинская диагностика, кредитный скоринг, прогноз оттока клиентов — требуют специфичных подходов (например, SHAP-значения, LIME), которые выходят за рамки данного курса.

Интерпретация предсказаний — отдельная задача, зависящая от предметной области: медицинская диагностика, кредитный скоринг, прогноз оттока клиентов — требуют специфичных подходов (например, SHAP-значения, LIME), которые выходят за рамки данного курса. **Метрика:** интерпретируемость модели не является частью математической оценки качества и не включается в метрики вроде accuracy, AUC-ROC или RMSE



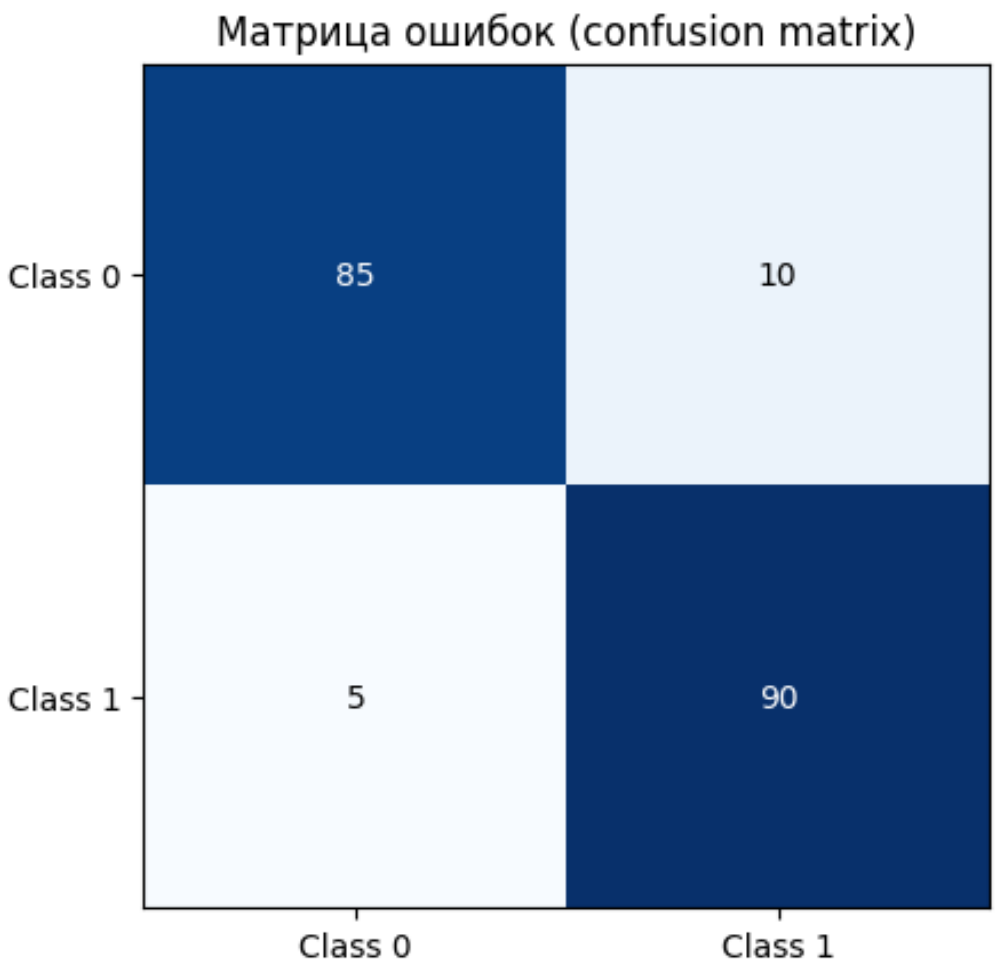
Кривые обучения loss accuracy по эпохам

. **Архитектура:** интерпретируемые модели — такие как метод ближайшего соседа (k-NN) — строятся на основе поиска похожих прецедентов в обучающей выборке: $y = \arg\max_{y_j} \mathrm{sim}(x, x_j)$



Scatter-диаграмма эмбеддингов вложение признаков

, где sim — функция схожести (например, cosine или L2). **Алгоритм:** объяснение через признаки предполагает анализ вкладов отдельных признаков в предсказание — например, через пороговые условия: если $f_i(x) > t_i$



Матрица ошибок confusion matrix

, то класс = 1. Такой подход применим только при наличии линейной или кусочно-линейной структуры модели и не масштабируется на нейросети без дополнительных инструментов (SHAP, Integrated Gradients).